Empirical Software Engineering (2024) 29:130
https://doi.org/10.1007/s10664-024-10502-3

Industrial adoption of machine learning techniques for early identification of invalid bug reports

Muhammad Laiq[1] · **Nauman bin Ali**[1] · **Jürgen Börstler**[1] · **Emelie Engström**[2]

Accepted: 17 May 2024

Abstract

Despite the accuracy of machine learning (ML) techniques in predicting invalid bug reports, as shown in earlier research, and the importance of early identification of invalid bug reports in software maintenance, the adoption of ML techniques for this task in industrial practice is yet to be investigated. In this study, we used a technology transfer model to guide the adoption of an ML technique at a company for the early identification of invalid bug reports. In the process, we also identify necessary conditions for adopting such techniques in practice. We followed a case study research approach with various design and analysis iterations for technology transfer activities. We collected data from bug repositories, through focus groups, a questionnaire, and a presentation and feedback session with an expert. As expected, we found that an ML technique can identify invalid bug reports with acceptable accuracy at an early stage. However, the technique's accuracy drops over time in its operational use due to changes in the product, the used technologies, or the development organization. Such changes may require retraining the ML model. During validation, practitioners highlighted the need to understand the ML technique's predictions to trust the predictions. We found that a visual (using a state-of-the-art ML interpretation framework) and descriptive explanation of the prediction increases the trustability of the technique compared to just presenting the results of the validity predictions. We conclude that trustability, integration with the existing toolchain, and maintaining the techniques' accuracy over time are critical for increasing the likelihood of adoption.

Communicated by: Bowen Xu, Xiaofei Xie, Maxime Cordy, and Bibi Stamatia

This article belongs to the Topical Collection: *Machine Learning Techniques for Software Quality Evaluation (MaLTeSQuE)*

✉ Muhammad Laiq
muhammad.laiq@bth.se

Nauman bin Ali
nauman.ali@bth.se

Jürgen Börstler
jurgen.borstler@bth.se

Emelie Engström
emelie.engstrom@cs.lth.se

[1] Blekinge Institute of Technology, Karlskrona, Sweden

[2] Lund University, Lund, Sweden

Published online: 31 July 2024

Keywords Invalid bug reports · Software quality · Software maintenance · Defect classification · Machine learning · Concept drift

1 Introduction

Users of software products report undesired behaviors of the products as bug reports. Companies address these bug reports to improve their products. The number of such bug reports may be high for large products. For instance, Mozilla received 224,408 bug reports in 12 months, and Firefox received 126,967 bug reports in 18 months (Fan et al. 2018). However, submitted bug reports may be invalid. Invalid bug reports are bug reports that do not describe deviations from the defined behavior of a system and are often related to misunderstandings of the system's functionality (Sun 2011). For Mozilla and Firefox, Fan et al. (2018) found that 31% and 77%, respectively, of the submitted bug reports were invalid.

When developers are assigned a bug report, they need to investigate if a corrective action in the code is required. To arrive at a suitable action, they need to understand the issue together with other stakeholders. This involves replication of the issue, consulting product documentation, or discussions with product owners and bug report authors. When a bug report assigned to a developer turns out to be invalid, a lot of resources have already been spent unnecessarily on its investigation. In Laiq et al. (2022), we found that 14–17% of bug reports assigned to developers turn out to be invalid. Automatically determining the likely validity of a bug report before assignment to developers may reduce the effort significantly.

Several studies (Zanetti et al. 2013; Fan et al. 2018) have used ML and deep learning (He et al. 2020) techniques for predicting the validity of bug reports with high predictive accuracy. However, these studies do not involve practitioners, use only archival bug reports data from open-source software systems, do not conduct evaluations in real settings, and have not yet been integrated into bug management processes or practices.

Despite the potential advantages of early identification of invalid bug reports in practice, the adoption of the proposed techniques in practice is yet to be investigated. *Thus, in this study, we use a technology transfer model* (Gorschek et al. 2006) *to guide the adoption of an ML technique for the early identification of invalid bug reports in practice.*

There may be several factors that affect the adoption of ML techniques (Rana et al. 2014b; Paleyes et al. 2022). In our previous work (Laiq et al. 2022), we found that it is critical for practitioners to understand the prediction results of the ML technique. However, the explainability aspect of ML techniques to determine the validity of bug reports has not been studied. *Thus, in this study, we investigate how the explainability of ML techniques for early identification of invalid bug reports can be supported by means of current approaches for interpreting the predictions of ML techniques.*

Another important factor for adoption is an ML technique's performance over time. When systems evolve over time, so will the bug reports since existing issues will be solved, new features will be added, and usage patterns will change. The performance of an ML technique will likely degrade when underlying concepts in the data change; this variation is known as concept drift (Ekanayake et al. 2012). Earlier research on bug prediction has indicated that the data used for bug prediction suffers from concept drift (Bennin et al. 2020; Kabir et al. 2020). In our study, this issue became apparent when the deployed model in the company could not match the accuracy level we achieved on archival data from the company. Thus, we decided to investigate the reasons of this mismatch. After ruling out any implementation issue that may explain poor performance, we investigated if concept drift may explain the mismatch.

In this paper, we present an industrial case study to evaluate the adoption of an ML technique in real industrial settings for the early identification of invalid bug reports. Our study provides new insights into the adoption of ML techniques in proprietary contexts as we go beyond the prediction accuracy achieved on archived data, which has been the case mainly in previous work (Fan et al. 2018; He et al. 2020).

The outline of this paper is as follows. Section 2 describes the related work on determining the validity of bug reports and adopting ML techniques in software development practice. We also describe how the current paper extends our previous work on the topic. Sections 3 and 4 present the research method of our study, i.e., the case description, data collection, and design and analysis iteration for each research question. In Section 5, we present the results of our study. In Section 6, we discuss our findings. Section 7 describes the validity threats to our study. Finally, in Section 8, we conclude the paper.

2 Related Work

In this section, we discuss the related work on the validity of bug reports and the adoption of ML techniques in software development practice and describe how we extend our previous work on the topic.

2.1 Bug Report Validity

There are few studies that have investigated the validity of bug reports. The studies (Zanetti et al. 2013; Fan et al. 2018; He et al. 2020) most relevant to our work are described below.

Zanetti et al. (2013) construct a collaboration graph capturing the connections between people working on a defect's resolution. They use two fields of bug reports from Mozilla Firefox, Mozilla Thunderbird, Eclipse, and Netbeans projects to identify a connection: (1) the field listing to whom a bug report is assigned and (2) the field listing other people who are tagged as relevant for the bug report resolution. From the resulting graphs, nine features (e.g., the clusters of highly connected people) were extracted and used to predict the validity of bug reports using a Support Vector Machine (SVM) classifier.

Building on Zanetti et al. (2013)'s technique, Fan et al. (2018) complemented the graph-based features with features extracted from the bug report's text and the submitter's past experience. They used 33 features as input to SVM and Random forest classifiers for predicting the validity of bug reports. Their technique considerably improved the prediction accuracy of invalid bug reports compared to Zanetti et al..

A shared limitation of the techniques by Zanetti et al. and Fan et al. is that the required features for prediction might not be available in the early stages of bug management. He et al. (2020) found that the fields required for creating (Zanetti et al. 2013)'s collaboration graph were unavailable for a newly submitted bug report even after three days. Similarly, Fan et al. (2018) extract collaboration-related features using comments on bug reports that will be unavailable for a newly submitted bug report. Thus, these two prediction techniques can not be used in practice in the early stages of bug management (He et al. 2020).

Therefore, He et al. (2020) only used summaries and descriptions of the bug reports. They applied a deep learning technique, convolutional neural network (CNN), in the OSS context. Their technique improved the accuracy of invalid bug reports prediction compared to Fan et al. (2018)'s technique. However, their technique may be prone to overfitting since they split data into training and testing sets comprising a fixed number of bug reports instead of having

incremental folds for evaluation. Furthermore, their technique could not outperform (Fan et al. 2018)'s technique for all projects.

These studies described above mainly focus on predictive accuracy, do not involve practitioners, only use archival bug reports, and do not conduct evaluations by use in real-world settings. In this study, we use (Gorschek et al. 2006)'s technology transfer model to guide the adoption of an ML technique in practice for the early identification of invalid bug reports, which involves various validation steps. Furthermore, we elicit practitioners' adoption concerns before and after the use of an ML-based tool through (Rana et al. 2014b)'s technology adoption framework.

2.2 Adoption of ML-Techniques in Software Development Practice

Several studies have focused on adopting ML techniques in practice to assist practitioners in software maintenance tasks such as bug assignment and prediction (Oliveira et al. 2021; Borg et al. 2022; Rana et al. 2014a; Aktas and Yilmaz 2020). However, to the best of our knowledge, no study has studied the adoption of an ML technique in practice for the early identification of invalid bug reports.

A recent survey by Paleyes et al. (2022) reviewed case studies and experience reports on the industrial adoption of ML techniques. They identified various challenges, such as the explainability of the ML technique's predictions, computational cost, integration, and updating of the ML model to handle concept drift. Rana et al. (2014b) identified similar adoption concerns of practitioners for ML techniques in the context of bug prediction. Moreover, Rana et al. (2014b) proposed a framework for the adoption of ML techniques in industry.

In this study, we utilize (Rana et al. 2014b)'s framework to identify the adoption concerns of practitioners for the ML technique for the early identification of invalid bug reports. We further investigated some solutions to address the challenges of explainability and concept drift, which were observed when ML techniques were used at the case company.

2.3 Extension of Our Previous Work

This paper significantly extends our previous work (Laiq et al. 2022), where we: (i) quantified and characterized the problem of invalid bug reports in a closed-source context, (ii) evaluated various ML techniques for predicting invalid bug reports using bug reports from an industrial setting, (iii) performed an initial validation in the industry (without the use of the technique in operation by practitioners) of the applied ML technique with practitioners, and (iv) identified the practitioners' adoption concerns for the ML technique using a technology adoption framework.

In this work, we (v) considered additional techniques, including XGBoost, CNN, and Bidirectional Encoder Representations from Transformers (BERT), (vi) explored additional features to improve the accuracy of the ML techniques, (vii) conducted an initial in-use evaluation of the applied ML technique in an industrial setting (by having practitioners use the ML technique in operations to predict the validity of newly arriving bug reports), (viii) addressed the explainability concern of practitioners regarding the ML technique, and (ix) investigated the implications of concept drift in the bug reports data that we use to train ML techniques for early identification of invalid bug reports.

Furthermore, we extend the work (i)–(iv) with additional data from the two products (P1, P2) used in our previous work and also with data from a new product (P3) and more responses

for identifying adoption concerns of practitioners, i.e., 34 responses, previously we had only three responses.

3 Research Method

The study's overall goal is to investigate the industrial adoption of an ML technique for the early identification of invalid bug reports.

To achieve this goal, we followed a technology transfer model (Gorschek et al. 2006) to introduce the use of ML techniques at a company. The model guided the engagement and collaboration with the case company (see Fig. 1). The iterative process it recommends has several confidence-building steps to increase the buy-in from the practitioners. Several design and analysis iterations were conducted during this collaboration with the case company. In these iterations, we followed the case study research guidelines (Runeson and Höst 2009) to answer the following research questions:

RQ 1. What are the characteristics of invalid bug reports in a closed-source context?
This research question aims to quantify and characterize the problem of invalid bug reports in the case company (step 1 in Fig. 1). We assessed the amount, lead time, priority, severity, origins, and handling of the bug reports.
Additionally, we also aim to collect practitioners' opinions regarding (a) the importance of early identification of invalid bug reports, (b) the importance of the tool support for early identification of invalid bug reports, and (c) the potential use cases of the tool support for identifying invalid bug reports.

RQ 2. How effective are existing techniques for predicting the validity of bug reports in a large-scale closed-source context?
This research question aims to identify ML techniques that yield the most promising results in a closed-source context (step 3 in Fig. 1). We investigate the accuracy of various ML techniques for predicting the validity of bug reports using past bug reports from several products at the case company. Furthermore, for one of the

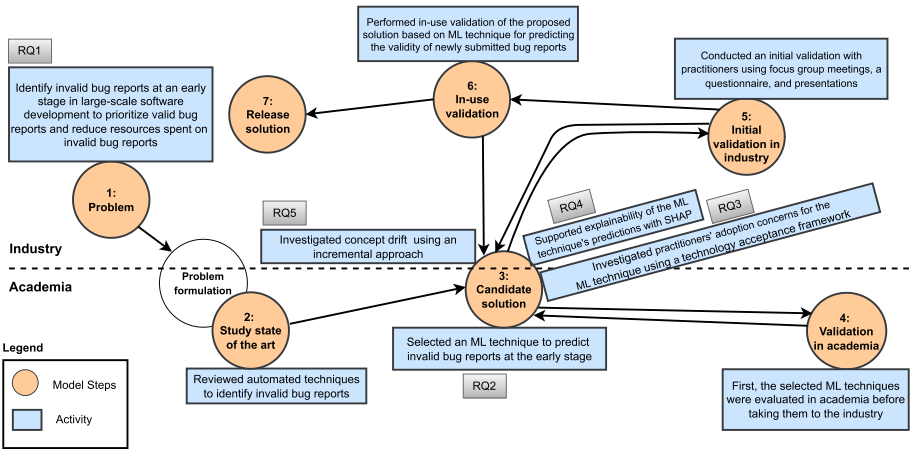


Fig. 1 An overview of industrial problems/challenges, activities, and research questions matching with the technology transfer model by Gorschek et al. (2006)

three products used in this study, the practitioners use an ML technique to predict the validity of newly submitted bug reports in live operation (step 6 in Fig. 1).

RQ 3. What factors are important for practitioners when deciding to adopt an ML technique for predicting the validity of bug reports?
This research question aims to identify important factors practitioners are concerned about while deciding to adopt an ML technique for determining invalid bug reports (step 5 in Fig. 1).

RQ 4. How can we support practitioners in interpreting the predictions of bug report validity?
In this research question, we explore how current approaches for explaining the prediction results of ML techniques may help practitioners understand and trust the predictions made by ML techniques regarding the validity of bug reports. The question became apparent during the initial validation of the chosen ML technique at the case company (i.e., step 5 in Fig. 1).

RQ 5. How large an issue is concept drift in bug reports data used for predicting the validity of bug reports?
In this research question, we aim to investigate the concept drift (Bennin et al. 2020) in bug reports data and how it may affect the ML techniques for predicting the validity of bug reports. The question became apparent during the operational use of the chosen ML technique (i.e., step 6 in Fig. 1).

3.1 Study Context

Figure 2 presents the context, case, and units of analysis (Runeson and Höst 2009). The case company is a large telecommunication vendor in Sweden, developing and maintaining large software products. It employs many individuals across multiple nations and specializes in developing embedded systems within the Information and Communications Technology (ICT) field. It uses a highly standardized development approach, e.g., TL9000 and ISO 9001 certified, and it complies with several standards set forth by IEEE, 3GPP, and ITU in its products. The development teams use agile practices and principles. The majority of the code at the company is written in C++, JavaScript, and Java.

At the case company, bug reports are filed from different sources, such as customers, testers, and developers. The change control board then reviews and assigns the submitted bug reports to a relevant development team. In the next step, developers use the submitted information to recreate the issue, identify root causes, and fix issues. However, in many

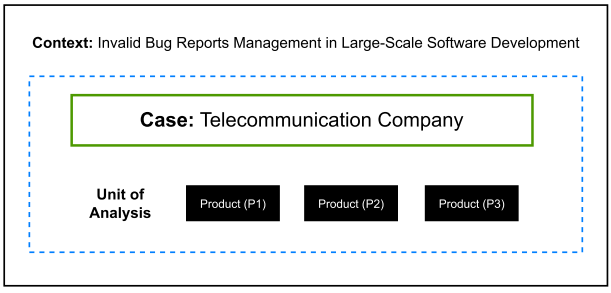


Fig. 2 Context, case, and unit of analysis

cases, bug reports do not describe erroneous system behavior, i.e., issues in the code. Such bug reports are resolved as invalid.

3.2 Operational Definition of Invalid Bug Reports

Similar to prior work (Zanetti et al. 2013; Fan et al. 2018; He et al. 2020), we defined invalid bug reports according to the working definition at the case company. Generally, invalid bug reports are bug reports that do not describe deviations from the defined behavior of a system. At the case company, invalid bug reports have one of the following resolution labels: no such requirements exist, configuration issues, misunderstanding of functionality, future requirements, and insufficient information.

Examples of valid and invalid bug reports: Below, we briefly discuss examples of valid and invalid bug reports reported in Bugzilla:

- **Invalid bug report** (https://bugzilla.mozilla.org/show_bug.cgi?id=1702998)
- **Heading**: *"Bookmarks Panel gets wide when opening the folder tree."* See the detailed description in Fig. 3. The description states that there is an issue with the bookmarks panel, i.e.,"*the panel should grow vertically to accommodate the folder tree, but not horizontally*." However, the answer to the submitted bug report clarifies that it is an expected behavior. Thus, the bug report is resolved as invalid. This issue might have been submitted due to a misunderstanding of the system functionality or a lack of knowledge about the feature.
- **Invalid bug report** (https://bugzilla.mozilla.org/show_bug.cgi?id=1392584)
- **Heading**: *"The hover state of the search field should match the spec."* See the detailed description in Fig. 4. The description states an issue with the hover state of the search field, as it does not match the specification. However, the answer to the bug report clarifies that this is not a bug (i.e., it is an expected behavior) but an issue with specification (i.e., documentation), and it will be updated. Eventually, the bug report was resolved as invalid because it described an issue with the documentation, not the code.
- **Invalid bug report** (https://bugzilla.mozilla.org/show_bug.cgi?id=1421432)
- **Heading**: *"Imported bookmarks don't show up in Highlights."* See the detailed description in Fig. 5. The description states an issue with the highlights for imported bookmarks. However, the answer to the submitted bug report clarifies that the bug is invalid; we actually get bookmarks, except for Safari, i.e., for Safari, we only import bookmarks, no

STR:

1. With Proton enabled, click on the Bookmarks button in the URL bar
2. Click on the expander next to the folder selector menulist

ER:

The panel should grow vertically to accommodate the folder tree, but not horizontally.

AR:

The panel grows both horizontally and vertically.

Fig. 3 An example of an invalid bug report reported in Bugzilla (id=1702998)

```
[Affected versions]:
Nightly 57.0a1

[Affected platforms]:
Platforms: Windows 10 x 64, Mac OS X 10.12 and Ubuntu 16.04 x64.

[Steps to reproduce]:
1. Launch Firefox.
2. Go to "about:preferences".
3. Verify the hover state of the search field.

[Expected result]:
The hover state of the search field border should be darker.

[Actual result]:
The hover state of the search field border appears the same as the normal state.
```

Fig. 4 An example of an invalid bug report reported in Bugzilla (id=1392584)

history, so no highlights will show up. This issue might have been submitted due to a misunderstanding of the system functionality or a lack of knowledge about the feature.

– **Valid bug report** (https://bugzilla.mozilla.org/show_bug.cgi?id=1719539)
– **Heading**: *"Pocket button can crash the browser in debug mode due to csp in about:pocket-saved."* See the detailed description in Fig. 6. The description states an issue with the pocket button, i.e., the pocket button can crash the browser in debug mode due to csp

```
Original comment with links here https://github.com/mozilla/activity-stream/issues/3753#issuecomment-
342857180

> The behavior differs depending on the browser we import from but overall it seems we are not affected.

>    for Chrome we don't import date created only url and title
>        we use current time & import history so higlights will show up.
>    for Edge we try to import date created and also import browsing history so everything should work fine
>    for Safari we only import bookmarks no history so no highlights will show up

> This is true for Windows as well (where it probably matters most).
> So we might get more bookmark highlights then we actually want from Chrome because using current date +
importing visits means all bookmarks are potential candidates.
> Not sure if it's something we necessarily want to fix/change. Leaving this open for comments.

> Some other things noticed:

>    no icons until you visit the website but we do fetch screenshots.
>    there is no description until you visit the websites

Bug is invalid we actually get bookmarks, except for Safari. Confirmed this with abenson.

Status: ASSIGNED → RESOLVED
Closed: 6 years ago
Resolution: --- → INVALID
```

Fig. 5 An example of an invalid bug report reported in Bugzilla (id=1421432)

Steps to test:

1. In an artifact debug build, start the browser
2. Browse to about:preferences. This causes the browser to “process flip” and load content in the parent process
3. Browse that same tab to about:pocket-saved
4. Browser should crash.

This was found via a new test added in ~~bug 1714749~~

Mike Conley helped me debug this one, and suspected it's due to about:pocket-* not being in this list: https://searchfox.org/mozilla-central/rev/94d6086481754e154b6f042820afab6bc9900a30/dom/security/nsContentSecurityUtils.cpp#954-967

Specifically about:pocket-saved is the issue because of this line https://searchfox.org/mozilla-central/source/browser/components/pocket/content/panels/saved.html#5

Doing it in debug more is the key to getting it to crash.

We didn't detect this until now, because of the test added in ~~bug 1714749~~

I believe we just need to add the about:pocket-* pages to the list in nsContentSecurityUtils.cpp

Fig. 6 An example of a valid bug report reported in Bugzilla (id=1719539)

in about:pocket-saved. The submitted bug report provides detailed steps for reproducing the bug and also suggests a solution to fix the issue. In response, the suggested solution is used to solve the issue, and finally, the bug is fixed with a status of “fixed” as the bug report is valid.

3.3 Study Participants, Investigated Products and Survey Participants

3.3.1 Study Participants

This study is conducted with the active collaboration of practitioners from the case company. We collected their feedback at various stages of the adopted technology transfer model (see Fig. 1) using focus group meetings, a questionnaire, and a presentation and feedback session with an expert. The profiles of the study participants are summarized in Table 1.

3.3.2 Studied Products

In this study, we investigated three products (P1, P2, and P3) of the case company. Detailed information about investigated products is shown in Table 2.

3.3.3 Survey Participants

We used convenience sampling to recruit respondents for the survey. In addition to the respondents from the case company, other respondents were invited from our contact networks. These respondents were asked to recruit more respondents. Using convenience sampling, we received 34 responses, of which 12 belonged to the case company.

As shown in Fig. 7(a), the participants of the survey are highly experienced in software development with diverse roles (e.g., developers and project managers, see Fig. 7(d)). The participants belong to companies of varying sizes (i.e., small, medium, and large based

Table 1 Profiles of the study participants from the case company, showing their experience in years (Exp), and their participation in focus group meeting-1 (FG1), focus group meeting-2 (FG2), focus group meeting-3 (FG3), and feedback session (FS)

Position	Description	Exp	FG1	FG2	FG3	FS
Manager	A product manager responsible for the development, communication, and product quality assurance.	∼20	✓	✓	✓	
Manager	A domain architect in Business Intelligence (BI) and analytics, responsible for the BI area within services delivery tools.	∼20	✓	✓		
Software Engineer & Quality Architect	An experienced software engineer responsible for managing the quality of two products at the case company.	∼10	✓	✓	✓	✓
Data Scientist	A data science team leader with a PhD in applied computing, his team is dedicated to building tools to boost R&D development.	∼10			✓	
Domain Expert	A software architect responsible for 'product release' in a cross-functional team.	∼8	✓	✓	✓	
Junior Data Scientist	A junior data scientist with a master's degree in software engineering.	∼2.5			✓	

on OECD (2023) criteria, see Fig. 7(b)), and most of them (28 out of 34) are familiar with ML for various reasons, e.g., they have tried ML using existing libraries (e.g., scikit-learn) in previous projects, see Fig. 7(c).

3.4 Data Collection

We collected data at various stages of the technology transfer model (see Fig. 1). Table 3 shows the collected data and its mapping to each research question (indicating the usage of data in answering questions). Furthermore, we also collected and reviewed the bug management-related documentation to understand how bug reports are managed at the case company.

Table 2 Profiles of the studied products and their bug report data from 2016–2022

Product	Product size	Code base age*	People	Distributed globally	Bug reports	% Invalid
P1	Large	>10 years	>100	Yes	∼3300	∼13%
P2	Medium	>10 years	>100	Yes	∼3400	∼16%
P3	Small	>8 years	>70	Yes	∼1000	∼18%

*All products are implemented using multiple programming languages, including mainly Java, JavaScript, Python, C++, and TypeScript

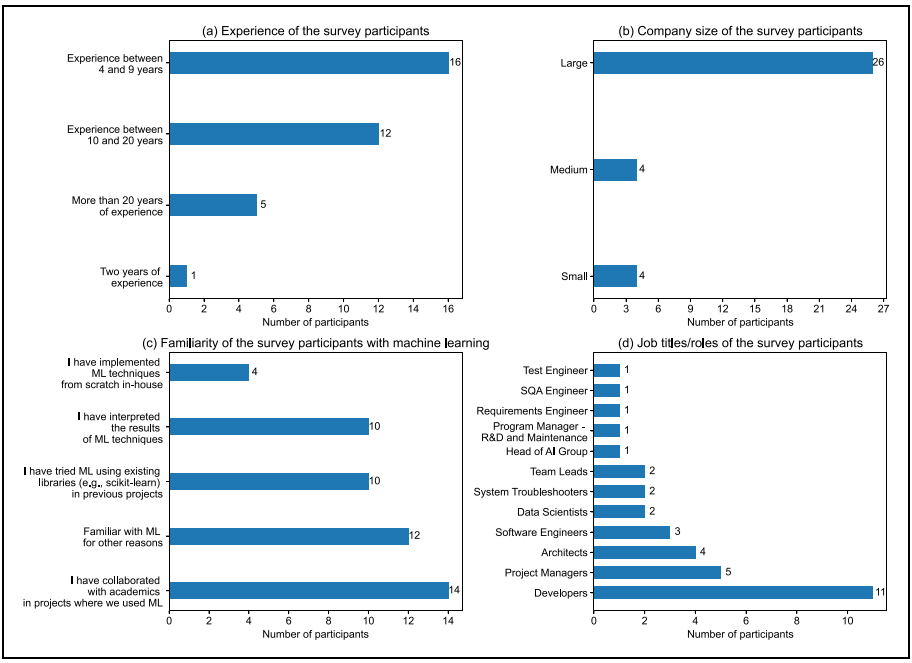


Fig. 7 Demographics information of the survey participants

Table 3 Data collection methods and their mapping to research questions (RQs)

Item	Description	Mapped RQs
Bug reports	We collected bug reports from the Bug Tracking System (BTS), a tool used by the case company in their bug management processes. In this study, we only use bug reports with a final verdict of being valid or invalid.	RQ1, RQ2, RQ4, and RQ5
Focus groups	We conducted three focus group meetings to present and collect practitioners' feedback on our work.	RQ1 and RQ2
Questionnaire	We used a questionnaire to identify practitioners' adoption concerns for the ML technique.	RQ1 and RQ3
Tool usage data	During operational use of the tool, we collected the expert's judgment about the validity of a newly submitted bug report and the prediction by the tool.	RQ2
Feedback session	We conducted a feedback session with a practitioner to evaluate the usefulness of the used explainability framework for explaining the ML technique's predictions.	RQ4

4 Design and Analysis Iterations

For each of the five research questions (see Section 3), we followed a design (including activities like technique implementation and data collection) and analysis iteration, which are described in detail in the remainder of the section.

4.1 Design and Analysis Iteration for RQ1 (What are the Characteristics of Invalid Bug Reports in a Closed-Source Context?)

We used descriptive statistics and graphs to understand the prevalence and characteristics of bug reports at the case company. We visualized the characteristics of invalid bug reports, such as their influx, grouping based on their origins, the average time from submission to resolution, priority, and mapping to software/system quality characteristics.

At first, the analysis mentioned above was done manually on a subset of bug reports, and the results were presented to practitioners in the first focus group meeting. Then, we automated the analysis as a diagnostic tool (a Python-based pipeline) because of the case company's interest. In the updated version of the diagnostic tool, we added support for a grouping of bug reports based on testing levels used at the case company. Later, the results were presented in the second focus group meeting.

Moreover, we used a questionnaire to collect practitioners' opinions about the importance of early identification of invalid bug reports and the need for tool support in this regard. We used a scale of importance ranging from "Essential" (must have), "Worthwhile" (nice to have), "Unimportant" and "Unwise" (detraction). We also asked the practitioners to provide potential use cases for a tool enabling the identification of invalid bug reports.

4.2 Design and Analysis Iteration for RQ2 (How Effective are Existing Techniques for Predicting the Validity of Bug Reports in a Large-Scale Closed-Source Context?)

To answer RQ2, we applied various ML techniques to bug reports data from three products (P1, P2, and P3) at the case company. The technology transfer model followed in this study (see Fig. 1) guided us to follow several validation steps described in the remainder of the section.

4.2.1 Validation in Academia

We identified ML techniques from the literature that may yield promising results in the selected case company. We selected Support Vector Machines (SVM), Random Forest (RF), Logistic Regression (LR), XGBoost (XGB), CNN, and BERT for the following reasons:

(a) We chose SVM and RF, as Fan et al. (2018) found that these techniques can effectively identify invalid bug reports. Although their research was conducted in the context of open-source software (OSS), their work is relevant as the features used in their work and ours are similar. We use the bug report's description and heading along with the submitter's validity rate, total bug reports count, and recent bug reports count. Similarly, we also extract and use the features from the description of bug reports, such as "has_steps_to_reproduce" and "has_attachment."

(b) We chose LR because of its simplicity as a classification model and its prior use and performance in a closed-source context for the automatic assignment of bug reports

to engineers at a large telecommunication company (Borg et al. 2022). Although the tasks differed (i.e., bug assignment to a relevant engineer vs. our task to identify invalid bug reports), we consider these findings relevant. We also used similar features to the ones used by Borg et al. (2022), including the bug report description, summary, and priority.

(c) We chose XGB because of its performance for similar classification tasks, e.g., requirements and email spam classification (Nguyen 2021; Mustapha et al. 2020).

(d) We chose CNN, as He et al. (2020) showed that CNN can achieve slightly higher performance compared to simple ML techniques proposed by Fan et al. (2018) for identifying invalid bug reports.

(e) We chose BERT because transformer-based models are considered state-of-art models for similar classification tasks, e.g., text classification (Sun et al. 2019) and sentiment classification (Wu et al. 2021; Biswas et al. 2020).

During validation in academia (step 4, Fig. 1), the selected techniques listed above were trained and tested on archival data from the case company. The implementation details (e.g., the used hyperparameters) of the above techniques can be found in Appendix A.

4.2.2 Initial Validation in Industry

After testing the techniques on archival data, we presented the results to the practitioners (step 5, see Fig. 1). We used two focus group meetings to collect the practitioners' feedback to improve the proposed solution.

Furthermore, we used a questionnaire to identify the concerns of practitioners when adopting an ML technique to determine the validity of newly submitted bug reports.

4.2.3 In-Use Validation

Based on the results of using the tool on archival data, the practitioners decided to test the tool in the limited scope of one product only (i.e., product P1). We handed over the tool to the practitioners managing bug reports for product P1 at the case company.

The deployed tool had the added functionality of recording the tool predictions for each bug report as well as the practitioners' judgments. For example, the data about a bug report with id #456 (not a real ID) contained the following information: predicted confidence level of valid: 88%, predicted confidence level of invalid: 12%, predicted class: valid, and practitioner's final judgment: valid.

We quantitatively analyzed the tool's performance on newly submitted bug reports in 2022 using the recorded information.

4.2.4 Feature Selection

Table 4 presents features we used in the prediction model of ML techniques. The feature selection is based on previous work, including (Laiq et al. 2022 and Fan et al. 2018).

These features listed can be calculated from a newly submitted bug report immediately. For example, features related to the submitter experience (F4, F5, and F13) can be derived based on the past submission history of the submitter. Similarly, F1, F2, and F6–F11 can be obtained from the description and heading of a bug report.

Unlike Fan et al. (2018), we do not use features related to readability, completeness, and collaboration network dimensions. As readability and completeness have negligible significance in distinguishing valid and invalid bug reports (Fan et al. 2018). Also, the collaboration

Table 4 Bug report features used for validity prediction

Id	Feature	Description
F1	Heading	A short description of a bug.
F2	Description	A detailed description of a bug, e.g., explaining the issue or describing steps to reproduce the issue.
F3	Priority	Priority of a bug at three levels (i.e., High, Medium, and Low).
F4	Submitter total bug count	The total number of bug reports submitted by a submitter in the past.
F5	Submitter validity rate	The validity rate of a submitter, i.e., the ratio of valid to total bug reports submitted by the submitter in the past.
F6	Has-stepstoreproduce	Indicates if the description of a bug report has steps to reproduce.
F7	Has-attachment	Indicates if the description of a bug report has an attachment.
F8	Has-testcase	Indicates if the description of a bug report has a test case.
F9	Has-code	Indicates if the description of a bug report has any code.
F10	Has-stacktrace	Indicates if the description of a bug report has stack trace.
F11	Has-patch	Indicates if the description of a bug report has a patch.
F12	Caught where	Testing level or phase where the bug was caught.
F13	Submitter recent bug count	The total number of bugs reported by a submitter in the last 90 days.

network is not available in the case company, which might not be available in the open source context as well in the early stages of bug management (He et al. 2020).

We calculate features F6–F11 using regular expressions from the description of a bug report. Our approach is similar to that used by Fan et al. (2018). For example, we employ regular expressions that use terms like "steps to reproduce" and "reproduce steps" to determine whether a bug report contains information about reproducing the issue. For detailed information on the extraction of these features, please refer to Fan et al. (2018)[1].

Submitter experience-related features (F4, F5, and F13) are calculated based on the historical data of an individual submitter; for example, F5: Submitter Validity Rate is a ratio of valid to total bug reports submitted by a submitter.

The F5: Submitter Total Bug Count feature does not take into account the impact of time on a submitter's experience. For instance, if a particular submitter has reported 100 bug reports, F5: Submitter Total Bug Count for his next bug report will be calculated as 100, regardless of when he submits his next bug report, which could be a year later. This will likely decrease the validity of the bug report submitted since the reporter may not have been familiar with the systems/processes. Thus, to cover the temporal evolution of the metrics related to the submitter, we use F13: Submitter Recent Bug Count, which aims to cover the aspect of whether the submitter has been active recently or not (that is, a 90-day window similar to Fan et al. (2018)).

4.2.5 Feature Extraction

In this section, we describe the process we followed for feature extraction.

[1] https://github.com/YuanruiZJU/TSE-Valid-Bug/blob/master/recognizing-technical-information.md

Textual Feature Extraction The standard data preprocessing is applied to textual features, including removing unnecessary data and stop words.

Unnecessary data includes whitespace and specific headings in the template used at the case company for describing a bug report. Stop words are commonly used words, such as the, we, you, etc., and they do not contribute to the prediction model.

After removing unnecessary data and stop words, we use: (a) a regular Term Frequency-Inverse Document Frequency (TF-IDF) term weighting scheme to convert our textual features into vectors. TF-IDF is a commonly used weighting scheme in the context of software engineering for text mining and information retrieval (Borg et al. 2014), (b) word2vec for word embedding using the Skip-gram algorithm, and (c) DistilBERT for word embedding. Transformer-based models are considered state-of-the-art for word embedding and can potentially achieve better accuracy in validity prediction than TF-IDF and word2vec.

TF-IDF is used with all the selected ML techniques, DistilBERT-based embeddings is used with the two top-performing ML techniques, and word2vec is used with CNN.

Categorical Feature Extraction We use the one-hot encoding technique to convert categorical features into vector form. In one-hot encoding, data for the categorical attribute is represented by a binary vector, where the length of the vector represents the number of possible values of the categorical attribute. In the binary vector, only a single place holds 1 for a particular data sample, and the remaining positions contain 0. For instance, if bug priority has categorical values [High, Medium, Low], a bug report with Medium priority would be coded as $< 0, 1, 0 >$.

Numerical Feature Extraction The numeric features are preprocessed using the StandardScaler technique for standardized input to the prediction model. StandardScalar is typically used to handle numeric data. It standardizes the data by subtracting the mean and dividing the standard deviation from each input, shifting the distribution to have approximately zero mean and one standard deviation.

4.2.6 Experimental Setup

There are various evaluation approaches, such as cross-validation, testing on fixed data sets, and time splitting or incremental. In this work, similar to Bhattacharya et al. (2012), Wang et al. (2014) and Bettenburg et al. (2008), we used an incremental approach. This approach reduces experimental bias (Witten and Frank 2002), and it is appropriate in our context since we aim to predict the validity of future bug reports. Therefore, we divided our dataset into 11 folds after sorting chronologically. Then, we performed our experiments as follows: In the first run, fold-1 was used for training and fold-2 for testing. In the second run, fold-1 and fold-2 were used for training and fold-3 for testing. This process was repeated ten times, and in the last run, fold-1 to fold-10 were used for training and fold-11 for testing. Finally, we measured the average prediction results over each fold.

4.2.7 Evaluation Metrics

We chose the Area Under the Curve (AUC) of receiver operator characteristic (ROC) (Huang and Ling 2005) for evaluating the performance of the models. AUC is unbiased and robust towards imbalanced datasets (Tantithamthavorn and Hassan 2018; Lessmann et al. 2008). AUC scores range from 0 to 1. The higher the value, the better the classification performance. An AUC of 0.5 is no better than a random guess for binary classifications.

The value of AUC is calculated by plotting the ROC curve using True Positive Rate (TPR) and False Positive Rate (FPR) for all thresholds. TPR is plotted on the y-axis and FPR on the x-axis.

$$TPR = \frac{TP}{TP + FN} \tag{1}$$

$$FPR = \frac{FP}{FP + TN} \tag{2}$$

The threshold value (i.e., a value in the range of 0 to 1) is compared with the predicted probability value. If the predicted probability value is higher than the threshold for a bug report, it is classified as valid and invalid otherwise. However, AUC is threshold-independent (Bradley 1997) and evaluates prediction performance against all thresholds.

Additionally, we measure Matthew's Correlation Coefficient (MCC) (Matthews 1975), recall, F1-score, and precision. The aim of measuring these metrics is to compare the reliability of underlying evaluation metrics (i.e., AUC) on imbalanced datasets.

MCC is suitable for evaluating imbalanced data sets (Halimu et al. 2019) and widely used in the biomedical field (Matthews 1975). MCC is a correlation coefficient between predicted and observed values of the confusion matrix and can be calculated from a confusion matrix. MCC values range from 1 to -1, where 1 shows perfect classification and -1 perfect misclassification. An MCC of 0 is no better than coin tossing.

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{3}$$

The normalized MCC (nMCC) can be calculated as follows:

$$nMCC = \frac{MCC + 1}{2} \tag{4}$$

$$Recall = \frac{TP}{TP + FN} \tag{5}$$

$$Precision = \frac{TP}{TP + FP} \tag{6}$$

$$F1 - score = 2 * \frac{Precision * Recall}{Precision + Recall} \tag{7}$$

4.3 Design and Analysis Iteration for RQ3 (What Factors are Important for Practitioners When Deciding to Adopt an ML Technique for Predicting the Validity of Bug Reports?)

To identify practitioners' concerns when deciding to adopt an ML technique for the early identification of invalid bug reports, we use the technology adoption framework proposed by Rana et al. (2014b). Our goal is to provide insight into the factors that practitioners prioritize when deciding to adopt ML techniques for identifying invalid bug reports, as well as to identify their main concerns.

The framework consists of nine factors. These factors further include sub-factors that are important for practitioners to assess prior to deciding to adopt (or not to adopt) an ML technique. The framework has been evaluated on an ML technique for software bug prediction (Rana et al. 2014a). Likewise, we identify and evaluate such aspects in our context,

i.e., an ML technique for determining the validity of bug reports. Table 5 shows the identified factors and their attributes. To understand the perceived importance of these factors, we used a web-based questionnaire to collect practitioners' opinions regarding their importance. We used a scale of importance ranging from "Essential" (must have), "Worthwhile" (nice to have), "Unimportant" and "Unwise" (detraction).

Table 5 Important factors and their attributes for the adoption of the ML technique for determining invalid bug reports, adopted from Laiq et al. (2022) and Rana et al. (2014b)

Factors	Attributes
Need and importance	Need for early identification of invalid bug reports
	Importance of having a tool based on ML technique for the early identification of invalid bug reports
Potential use-cases of tool	UC1: Additional review for likely invalid bug report before assignment
	UC2: Deprioritization of likely invalid bug reports
	UC3: Flagging invalid bug reports for down-stream developers
Familiarity and experience with ML	Understanding of ML-based technology
	Collaborated with academics in ML-based projects
	Tried ML in previous projects
	Ability to interpret results of ML technique
	Ability to assess the quality of ML technique's results
	Ability to implement ML techniques in-house
Other important aspects, i.e., perceived benefits and barriers and tool availability	Accuracy of the predictions by ML technique
	Explainability of the ML technique
	Adaptability to different product units such as P1 and P2
	Ability to handle a large amount of bug reports data
	Integration in the current workflow
	Integration with existing systems
	Compatibility with existing systems
	Cost for data collection
	Cost for training and education of people
	Cost for installation
	Cost for licensing
	Cost for computational resources
	Cost for maintenance
	Training time by the ML technique
	Learning curve
	Buy-in from the concerned stakeholders
	Availability as an open-source tool
	Availability of technical support

4.4 Design and Analysis Iteration for RQ4 (How Can we Support Practitioners in Interpreting the Predictions of Bug Report Validity?)

When we presented the evaluation results of ML techniques for validity prediction at the case company, the need for explainability became apparent, i.e., practitioners wanted to know the underlying reasons behind predicting a bug report as valid or invalid.

To support the explainability of ML techniques, the frameworks such as Local interpretable model-agnostic explanations (LIME) by Ribeiro et al. (2016) and SHapley Additive exPlanations (SHAP) by Lundberg and Lee (2017) could be utilized. However, LIME is limited to local explainability (i.e., it can only be used for explaining the prediction of a specific instance). SHAP provides support for both local and global explanations (i.e., it describes the overall process of how an ML model works). Furthermore, SHAP is more accurate and theoretically sound compared to LIME (Linardatos et al. 2020). Thus, in this study, we use the SHAP framework by Lundberg and Lee (2017) to explain the prediction results and features that differentiate valid and invalid bug reports.

We conducted a feedback session with a practitioner (Software Engineer & Quality Architect, see Table 1) to evaluate the usefulness of the SHAP framework for explaining the prediction of the ML technique. We conducted our feedback session as follows:

- At first, the selected practitioner was given brief information about how we predict the validity of newly submitted bug reports.
- Then we presented one of the examples to the practitioner, i.e., explaining the prediction results for a bug report.
- Afterward, the practitioner was asked to read the SHAP explanations for the nine randomly selected instances to assess whether the SHAP explanations helped understand those predictions (i.e., prediction of those instances as valid or invalid bug reports). Random selection was performed to avoid selection bias.
- Finally, we collected feedback from the practitioner based on the interpretations of the nine predictions in the previous step. We used the following question for collecting feedback on each sample: *On a scale of 1 to 5, how helpful was the explanation for understanding the prediction made by the ML technique? One being not at all helpful and 5 being very helpful.*

4.5 Design and Analysis Iteration for RQ5 (How Large an Issue is Concept Drift in Bug Reports Data Used for Predicting the Validity of Bug Reports?)

When the model was deployed in the company, it could not match the accuracy level we achieved on archival data from the company. Thus, we investigated the reasons for this mismatch. After ruling out any implementation issue that may explain poor performance, we investigated if concept drift may explain the mismatch.

To identify concept drift in our bug reports data, we followed the same hypothesis as used in previous work (Klinkenberg and Joachims 2000; Last 2002; Nishida and Yamauchi 2007), i.e., if the performance of an ML model degrades (in terms of prediction error rate) on the current window (i.e., bug reports data of 90 days) compared to the older window, there is concept drift; otherwise, there is no concept drift.

As shown in Fig. 8, similar to previous work (Zliobaite 2010; Kabir et al. 2019), we used an incremental window-based approach with a full-memory training strategy. Similar to previous work (Li et al. 2020; Xu et al. 2018), we chose a window size of 90 days, i.e., bug reports data of 90 days. We conducted our experiments as follows:

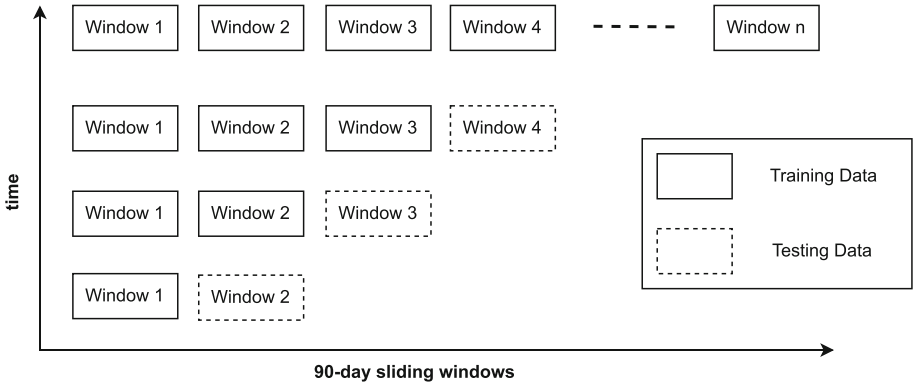


Fig. 8 Full-memory 90-day sliding windows, following Klinkenberg and Joachims (2000); Zliobaite (2010)

– At first, we trained our ML model on 67% of the data belonging to an old window (w_o) and then tested it on (a) the remaining 33% from the old window and (b) the data from a recent window (w_r).
– We then calculated the statistical difference of the model prediction error rate on w_o and w_r.
To measure statistical significance, we used a strategy similar to de Lima Cabral and de Barros (2018), i.e., the significance of concept drift between old and recent windows using Fisher's exact test. Fisher's exact is relevant in our context as it is not affected by sample size, data imbalance, or data sparsity (Mehta 1994).
The 2x2 contingency table of old and recent windows for Fisher's exact is calculated as follows:

$$Errors = [w_r, w_o] \tag{8}$$

$$Hits = [w_r, w_o] \tag{9}$$

– Finally, we calculated the p-value. If it is less than 0.05, we reject the null hypothesis (i.e., the two windows are of the same type) and consider the alternative hypothesis that there is a difference between the distribution of the two windows, i.e., the presence of concept drift.

5 Results and Analysis

5.1 RQ1: What are the Characteristics of Invalid Bug Reports in a Closed-Source Context?

A diagnostic tool was used to analyze invalid bug reports' characteristics in terms of their prevalence, lead time, and priority. In Table 6, we present the lead time and median priority of valid (V) and invalid (I) bug reports for all three products. For P1, P2, and P3, we get around 18, 16, and 13 mean calendar days, respectively. The average lead time of invalid bug reports is similar to valid bug reports, and the average priority of three products for invalid bug reports is medium (B). This signifies the need to address the challenge of early identification of invalid bug reports at the case company.

Table 6 Lead time (mean calendar days from submission to resolution) and median priority (A: High, B: Medium, C: Low) for valid (V) and invalid (I) bug reports

Product	Lead time (V)	Lead time (I)	Priority (V)	Priority (I)
P1	∼15	∼18	C	B
P2	∼19	∼16	B	B
P3	∼15	∼13	B	B

Similarly, we analyzed the origin of invalid bug reports for all three products (P1, P2, and P3) by mapping bug reports to their corresponding testing levels used at the case company. However, we observed that most of the faults slip through to later stages (i.e., verification or verification on the target environment). Likewise, most of the software/system quality characteristics were mapped to similar categories, i.e., functional correctness and functional completeness.

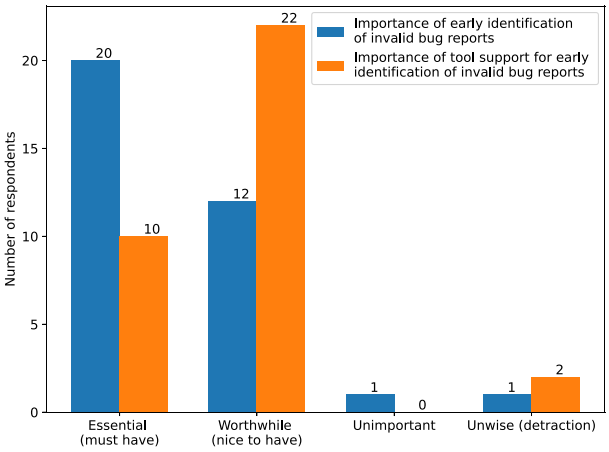
For products P1, P2, and P3, 18%, 16%, and 13% of the bug reports were invalid, respectively. Over the years, P1 has improved a bit in terms of reducing the percentage of invalid bug reports. However, P2 and P3 have not changed much. Fan et al. (2018) reported 31% and 77% of invalid bug reports for the open-source projects Mozilla and Firefox, respectively. In contrast to open-source, the percentage is smaller. A potential reason for that could be the rigorous processes for bug writing being followed in a proprietary context and the discrepancy of skills and motivation of the bug submitters (Bachmann and Bernstein 2009).

In addition to quantitative analysis of the characteristics of invalid bug reports, we also collected practitioners' opinions regarding (a) the importance of early identification of invalid bug reports, (b) the importance of tool support for early identification of invalid bug reports, and (c) the potential use cases of the tool support for identifying invalid bug reports. Below, we summarize the results of their responses.

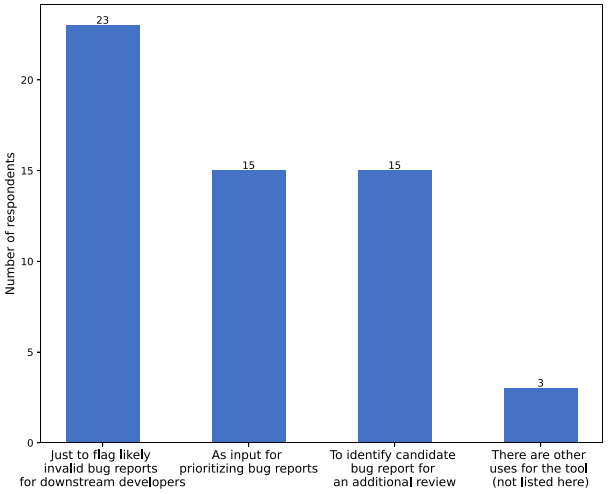
Importance of early identification of invalid bug reports and tool support: Figure 9(a) shows that overall, the respondents considered the early identification of invalid bug reports as important. 20 of the 34 respondents considered early identification of invalid bug reports essential, and 12 considered it worthwhile, i.e., nice to have.

However, regarding tool support for identifying invalid bug reports, only 10 of the 34 respondents considered it essential, and a majority, 22 of the 34 respondents, considered it worthwhile.

Only two respondents considered the early identification of invalid bug reports as "Unimportant" and "Unwise". The same respondents considered tool support for the early identification of invalid bug reports as "Unwise". However, we noticed discrepancies in their rating and their responses to other questions. One of the two respondents who rated the early identification of invalid bug reports as unimportant rated having a tool to support the early identification of invalid bug reports as worthwhile. Also, in the free text field, the same respondent states that the tool could be useful in preventing the registration of such bug reports in the first place. The second respondent, who had rated early identification of invalid bug reports as unwise, states in the free text field that an ML tool could be used to provide feedback to the author of a bug report before it is registered in the system. Similarly, in the free text field, the respondents state that the tool could be useful during bug assignment and in preventing the registration of such invalid bug reports.



(a) Importance of early identification of invalid bug reports and importance of tool support for early identification of invalid bug report



(b) Potential use cases of the tool support for identifying invalid bug reports

Fig. 9 Survey results about the importance of early identification of invalid bug reports and tool support and potential use cases of the tool support for identifying invalid bug reports

Potential use cases of the tool support for identifying invalid bug reports: As shown in Fig. 9(b), out of 34 respondents, 23 chose "flag likely invalid bug reports for downstream developers" as a potential use case of the tool. Another 15 respondents chose "as input for prioritizing bug reports" and "identify a candidate bug report for an additional review" as potential use cases for the tool. On further analysis of the survey responses, we found that a total of 31 respondents out of 34 agreed with at least one of the three potential use cases of the tool listed in the figure. Additionally, three respondents suggested a new use case for the tool, which is to use it to receive feedback when registering (reporting) the bug reports.

5.2 RQ2: How Effective are Existing Techniques for Predicting the Validity of Bug Reports in a Large-Scale Closed-Source Context?

In the following subsections, we present the results of the validation steps that were guided by the technology transfer model followed in this study (see Fig. 1).

5.2.1 Results of Validation in Academia

In Table 7, we present the results of the applied ML techniques using (a) the features F1–F5 and (b) the complete list of features that include newly added features (F6–F13), see Section 4.2.4 for details about feature selection. In Figs. 10, 11 and 12, we show the AUC values for each ML technique applied to three studied products over different folds.

Our results are comparable to previous work in an open-source context (Fan et al. 2018; He et al. 2020). Using the newly added features on P1, the Logistic Regression (LR) model achieved an AUC of 0.845, compared to 0.851 AUC of the old LR-based model, i.e., less than the old model but not much difference. Likewise, the performance of the Random Forest, SVM, and XGB classifiers has not changed much.

Generally, XGB is known for its superior performance compared to LR. However, our results indicate that the performance of XGB is slightly lower than that of the LR. A possible explanation for this could be data imbalance. Shahri et al. (2021) also reported that LR performed better than XGB on imbalanced datasets.

Table 8 shows the results of the top-performing simple ML technique (i.e., LR with TF-IDF) and its comparison with CNN, fine-tuned BERT and models using DistilBERT-based embeddings. The models based on DistilBERT embeddings and CNN achieve slightly higher AUC than simple ML techniques with TF-IDF. However, the fine-tuned BERT-based models notably outperform all other models.

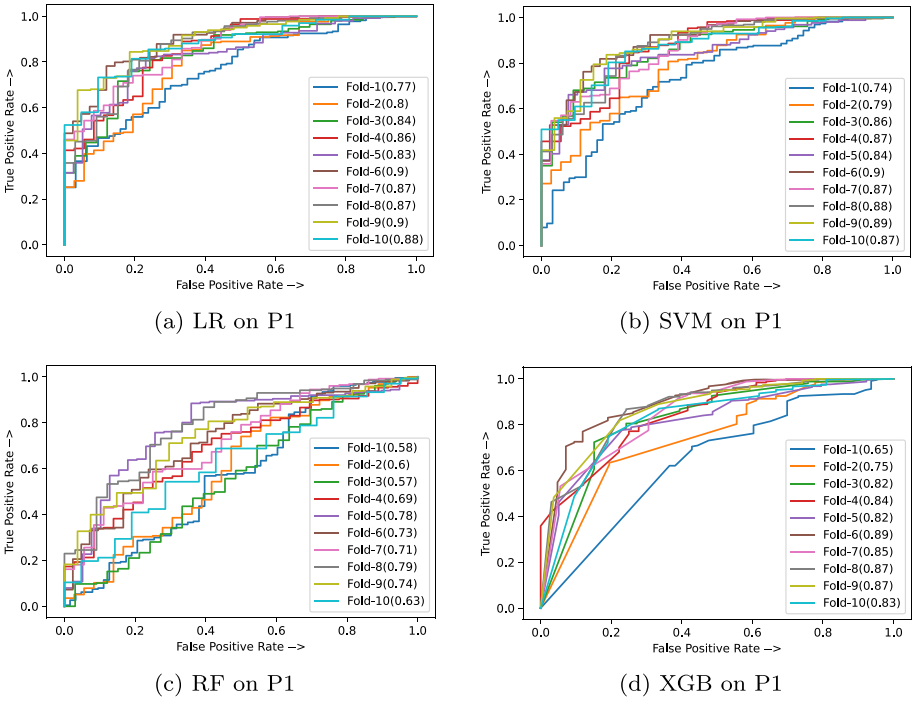
5.2.2 Results of Initial Validation in Industry

We conducted a focus group meeting with practitioners and presented the results of the ML techniques. The practitioners' overall response was positive. However, in the feedback,

Table 7 Model performance (area under the curve = AUC) on features (F1–F5) vs. F1–F13 using logistic regression (LR), support vector machines (SVM), random forest (RF), and XGBoost (XGB)

Product	Classifer	Features (F1–F5) AUC	Features (F1–F13) AUC
P1	LR	**0.851**	0.845
	SVM	**0.849**	0.835
	RF	0.681	**0.682**
	XGB	**0.817**	**0.817**
P2	LR	**0.765**	0.757
	SVM	**0.753**	0.716
	RF	0.619	**0.627**
	XGB	**0.739**	**0.739**
P3	LR	**0.840**	0.834
	SVM	**0.839**	0.834
	RF	0.627	**0.638**
	XGB	0.813	**0.816**

Top AUC values are highlighted in boldface



(a) LR on P1 (b) SVM on P1

(c) RF on P1 (d) XGB on P1

Fig. 10 AUC over different folds for each ML technique applied on P1

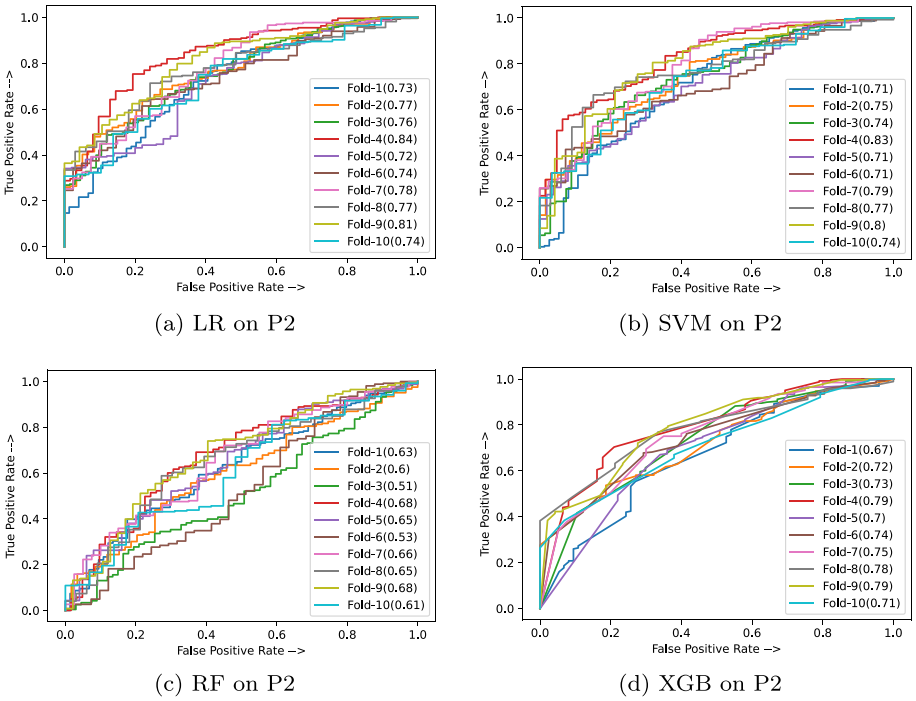
practitioners asked for providing explainability support with the predictions, i.e., underlying reasons behind predicting a bug report as valid or invalid.

Furthermore, it was decided to proceed with Logistic Regression, which was the best-performing among the simple ML techniques compared in the study. Although the Fine-tuned BERT-based model produced better results than LR (TF-IDF), we decided to use simpler techniques due to practitioners' preferences (see detailed discussion in Section 6.6) and the similar results we achieved using LR (TF-IDF), see Table 8.

Likewise, most practitioners (i.e., 4 out of 5) suggested not applying any class-rebalancing techniques to handle data imbalance (i.e., our dataset is skewed towards the valid class of bug reports). Because class-rebalancing techniques tend to produce false positives (Tantithamthavorn et al. 2018), i.e., they will misclassify valid bug reports in our context. Their collective feedback emphasized the importance of avoiding the misprediction of valid bug reports as invalid, as it would have significant consequences in their context.

The practitioners responded that the biased prediction for the valid class is acceptable for them (i.e., the prediction of invalid bug reports as valid), while the prediction of valid bug reports as invalid should be handled carefully. Thus, we did not add penalties for misclassifying the invalid bug reports class and proceeded for in-use validation with these settings, i.e., class_weight hyperparameter of the Logistic Regression was set to default value.

Based on the feedback during the initial validation in the industry, we improved the ML-based tool as follows: (a) the use of SHAP to support explainability, (b) the improvement of the user interface (UI) of the ML-based tool, i.e., the addition of UI that allows users to upload data and make predictions using the existing model. To ensure the up-to-date use of recent data, including submitter metrics, we added support for uploading the latest data



(a) LR on P2 (b) SVM on P2

(c) RF on P2 (d) XGB on P2

Fig. 11 AUC over different folds for each ML technique applied on P2

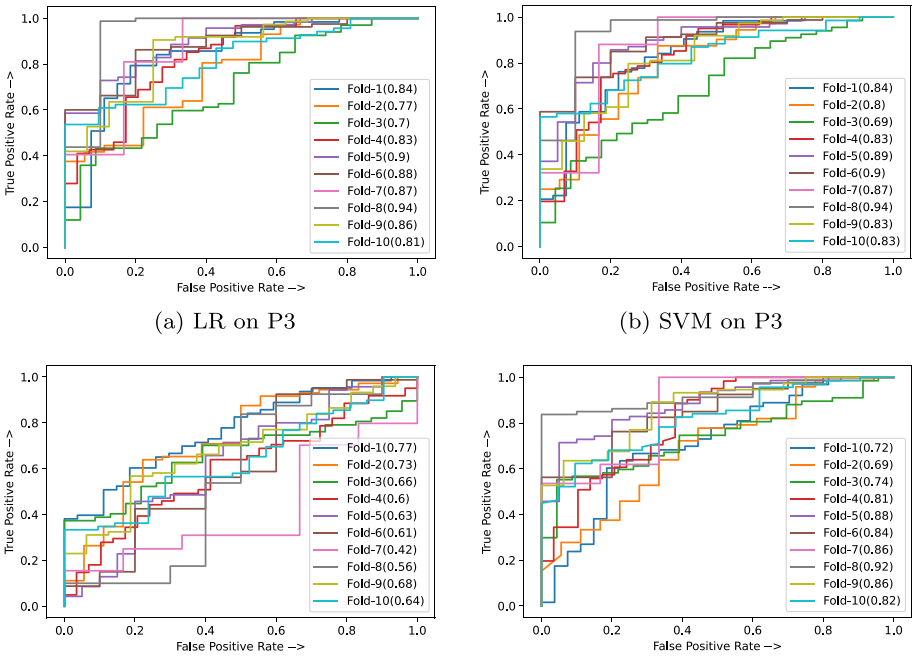
and retraining the model before making a prediction. We also added support to display the prediction confidence level for valid or invalid bug reports. Additionally, some improvements were made to the diagnostic tool used to analyze the characteristics of valid and invalid bug reports that we incorporated in the updated version of the tool (see details in Section 4.1).

5.2.3 Results of In-Use Validation

We developed a prototype tool implementing the ML technique selected by practitioners. The tool was trained on data from 2016 to 2021 and was handed over to the company for operational use to assess the validity of newly submitted bug reports in 2022. In this section, we will present the results of the in-use validation of the tool.

Based on our quantitative analysis, we found that our tool correctly classified 110 newly submitted bug reports out of 153 (130 valid and 23 invalid bug reports), which is **72%** accuracy. The results of other metrics were as follows: AUC: 0.78, MCC: 0.267, nMCC: 0.63, Invalid F1-score: 0.39, Invalid Precision: 0.29, Invalid Recall: 0.60, Valid F1-score: 0.82, Valid Precision: 0.91, and Valid Recall: 0.74. The tool's performance has decreased during the in-use validation compared to validation in academia, i.e., 0.78 AUC compared to 0.851 AUC. This prompted us to look for any changes that might have impacted the performance of the ML technique, such as product, technology, process, or organization.

On further analysis, we found that there were more than 20 new bug report submitters during 2022 in the investigated product (P1). Although they were not new at the case company, but their total bug count and validity rate features (F4 and F5, see Table 4) were computed



(a) LR on P3 (b) SVM on P3

(c) RF on P3 (d) XGB on P3

Fig. 12 AUC over different folds for each ML technique applied on P3

Table 8 Comparison of the top-performing simple ML technique (LR with TF-IDF) with DistilBERT-based embeddings on the two top-performing simple ML techniques (LR and SVM), CNN, and Fine-tuned BERT using features F1–F5: F1-score (F1), area under the curve (AUC), precision (P), and recall (R) for valid (V) and invalid (I) bug reports

Product	Classifier	AUC	F1(I)	P(I)	R(I)	F1(V)	P(V)	R(V)
P1	LR (TF-IDF)	0.851	0.411	0.705	0.305	0.942	0.906	0.982
	CNN (word2vec)	0.851	0.351	0.744	0.239	0.936	0.892	0.985
	LR (DistilBERT)	0.853	0.391	0.661	0.280	0.936	0.898	0.978
	SVM (DistilBERT)	**0.854**	0.491	0.382	0.690	0.881	0.945	0.826
	Fine-tuned BERT	**0.892**	0.267	0.167	0.898	0.936	0.994	0.885
P2	LR (TF-IDF)	0.765	0.247	0.538	0.168	0.910	0.857	0.970
	CNN (word2vec)	**0.775**	0.209	0.755	0.128	0.912	0.847	0.988
	LR (DistilBERT)	0.770	0.249	0.590	0.160	0.909	0.850	0.978
	SVM (DistilBERT)	0.769	0.237	0.636	0.147	0.911	0.849	0.984
	Fine-tuned BERT	**0.819**	0.128	0.072	0.799	0.909	0.993	0.839
P3	LR (TF-IDF)	0.840	0.530	0.725	0.448	0.902	0.872	0.938
	CNN (word2vec)	0.846	0.433	0.709	0.344	0.906	0.858	0.963
	LR (DistilBERT)	0.856	0.462	0.770	0.341	0.912	0.859	0.973
	SVM (DistilBERT)	**0.858**	0.367	0.878	0.240	0.913	0.845	0.992
	Fine-tuned BERT	**0.904**	0.346	0.218	0.967	0.914	0.998	0.842

The AUC values of the top two techniques for each product are highlighted in boldface

as zero, as no prior bug reports were submitted by them for P1. As the model treated these submitters as new employees, it impacted the accuracy of the ML technique.

Together with the practitioners, we decided to set the validity rate to 0.5 for people who are new to the product but have some relevant experience, and 0 for those who are new and do not have an understanding of their systems. The rationale behind this suggestion was that individuals who understand their systems are less likely to submit invalid bug reports than those who are new and have limited knowledge. With this modification, the ML technique could correctly classify bug reports with **0.82%** accuracy. The practitioners' feedback was positive on the tool's performance, and they acknowledged that the accuracy was reasonable.

5.3 RQ3: What Factors are Important for Practitioners When Deciding to Adopt an ML Technique for Predicting the Validity of Bug Reports?

We now present the results of the practitioners' opinion survey regarding the importance of various adoption factors that practitioners consider when deciding to adopt an ML technique for predicting the validity of bug reports (see summary in Fig. 13). The survey had 34 responses. Out of the 34 responses, 12 belong to the case company, which could potentially impact the overall trend. However, we analyzed those 12 and the remaining 22 responses separately and found the same trend.

Two out of 34 practitioners consistently rated most adoption factors as "Not at all important" or "Slightly unimportant." The responses count highlighted in red in Fig. 13 are mainly from those two practitioners. It would have been interesting to know the rationales behind such a rating, but unfortunately, we cannot get in touch with those two practitioners as they did not provide their email addresses.

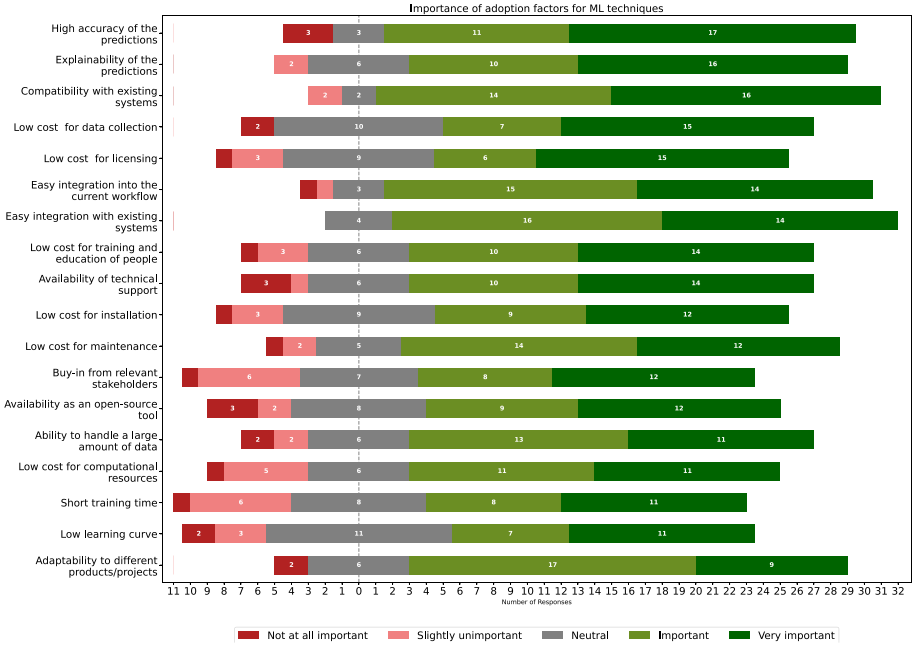


Fig. 13 Importance of adoption factors for ML techniques

Among the adoption factors that practitioners rated, the high accuracy of the predictions was rated as "Very important" by the highest number of practitioners (i.e., 17), followed by the explainability of the predictions and compatibility of the ML-based tool with existing systems. Both of them were rated as "Very important" by 16 practitioners.

Among the other adoption factors, the low cost of data collection and licensing was rated as "very important" by 15 practitioners, followed by easy integration into the current workflow, easy integration with existing systems, low cost for training and education of people, and availability of technical support, all of them were rated as "Very important" by 14 practitioners.

Of the remaining adoption factors, the low cost for installation, low cost for maintenance, buy-in from relevant stakeholders, and availability as an open-source tool were rated as "Very important" by 12 practitioners, followed by the ability to handle a large amount of data, low cost for computational resources, short training time, and low learning curve, which were rated as "Very important" by 11 practitioners. The adaptability to different products/projects was rated as "Very important" by nine practitioners.

While analyzing the survey results, we found several noteworthy findings described below: (a) Two practitioners rated the explainability of predictions as "Very important." However, they rated high prediction accuracy as "Not at all important." This may indicate that explainability is more important than accuracy for them (b) Another practitioner considers adaptability to different products/projects as "Not at all important," while accuracy and explainability were rated very important by that practitioner. (c) Among the others, similar trends were observed for several practitioners. For example, three practitioners did not consider the tool's availability as an open-source important but rated all other factors as "Important" or "Very important" in general.

Qualitative feedback of practitioners regarding the important adoption factors: In addition to the factors shown in Fig. 13, we also collected feedback from practitioners in a free text field about other important adoption factors for the ML-based tool for identifying invalid bug reports. Below, we list the additional factors that practitioners highlighted in the comments:

- The ML-based tool should have a self-explanatory user interface.
- The ML-based tool should be easy to configure.
- The usability (not user interface but the ease of use) of the ML-based tool is key to the adoption, i.e., it should be easy to use.
- The ML-based tool for identifying invalid bug reports should be simple and not disrupt the workflow while providing developers additional useful information (i.e., the validity of bug reports).
- The ML-based tool should be designed to assist support staff in their work without causing frustration to customers, i.e., the tool should carefully handle false predictions of invalid bug reports.

5.4 RQ4: How Can we Support Practitioners in Interpreting the Predictions of Bug Report Validity?

To address the practitioners' feedback regarding the need for explanations of predictions, we added an explainability aspect using the SHAP framework by Lundberg and Lee (2017). The SHAP value represents how much each feature contributes to an ML technique's prediction on average (i.e., the average marginal contribution of a feature), considering all possible combinations of features (Molnar 2023). In other words, the SHAP value indicates whether

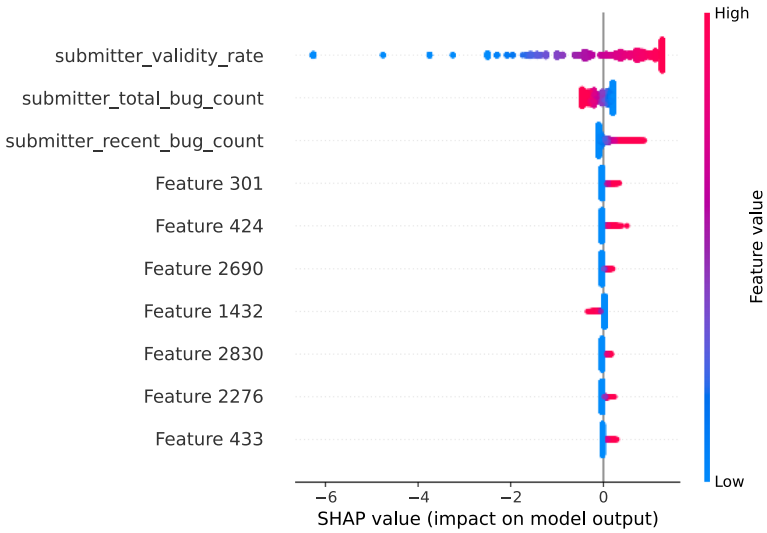


Fig. 14 Feature importance using bee swarm plot for logistic regression-based model, to maintain data confidentiality, the names of some company-specific features are anonymized

or not a feature increases the ML technique's prediction over a random baseline (Lundberg and Lee 2017).

In our binary classification problem (i.e., valid/invalid bug report), we want to predict valid and invalid bug reports. The valid class is positive and the invalid class is negative. By default, our ML technique attempts to predict a valid class. Thus, a positive SHAP value increases prediction towards a valid class (i.e., a valid bug report). Likewise, a negative SHAP value increases prediction towards an invalid class (i.e., an invalid bug report).

In Figs. 14, 15, 16, 17, 18 and 19, we summarize our analysis of the ML techniques' interpretation using the SHAP framework by Lundberg and Lee (2017). Figure 14 shows the top 10 most important features[2] using a bee swarm plot for the Logistic Regression model. The features are sorted according to their importance on the y-axis, and their SHAP values for each instance of training data are visualized using the bee swam plot on the x-axis. The red color indicates a high feature value, and the blue color indicates a low feature value. Here, we can see that the submitter validity rate is the most important feature in the Logistic Regression model. We can see the same in Fig. 15 for the XGBoost model. This seems quite rational and is in line with the literature (Fan et al. 2018; Laiq et al. 2022), i.e., people with high validity rates are likely to submit valid bug reports and vice-versa. Likewise, we can see that the submitter's total bug count is also an important feature in Logistic Regression and XGBoost models.

To demonstrate our findings, we further explain one valid and one invalid bug report using the SHAP force and waterfall plots; see Figs. 16–19. In Figs. 16 and 17, we can see two SHAP force plots depicting how different features contribute to the ML technique's predictions for predicting the two selected bug reports as valid/invalid. The features in red contribute towards a positive class (i.e., valid bug report) and their length shows their importance. We can see in Fig. 16 that the submitter validity rate is the biggest contributor to predicting the selected bug report as valid (i.e., the submitter of this bug report has a 95% validity rate). Similarly, In

[2] Due to the case company's anonymity requirement, the names of some features are anonymous.

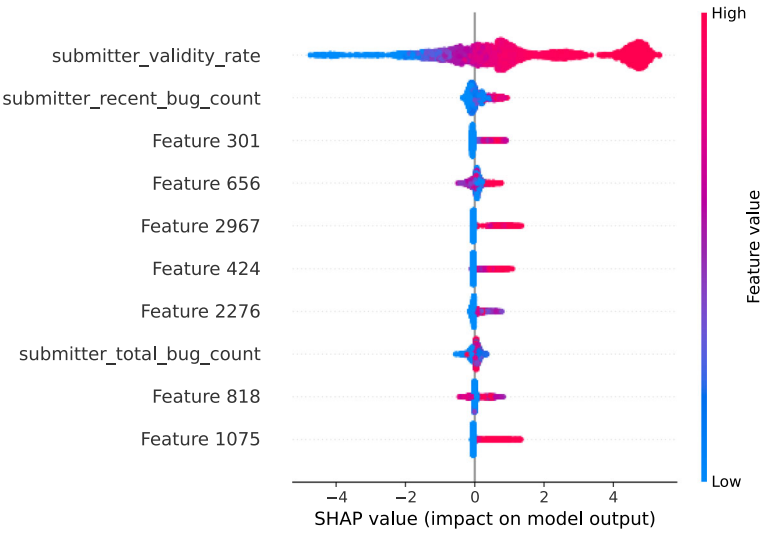


Fig. 15 Feature importance using bee swarm plot for XGBoost-based model, to maintain data confidentiality, the names of some company-specific features are anonymized

Fig. 17, we can see that the submitter validity rate feature contribution is high compared to other features for predicting the selected bug report as invalid. Likewise, in Figs. 18 and 19, we can see two plots picturing the contribution of each feature using the SHAP waterfall plot for predicting the two selected bug reports as valid/invalid. The negative values indicate the probability of less than 0.5 for a valid class. The bottom of the waterfall plot starts with the expected value of the model output (e.g., E[f(x)] = - 0.332 in Fig. 18), and then each row shows how the contribution of each feature for valid (red) and invalid (blue) classes changes the value from the expected model output using the background training data for the selected prediction, i.e., f(x) = 2.72, see Fig. 18. These findings are also in inline with our summary plots for both Logistic Regression and XGBoost model, see Figs. 14 and 15.

We conducted a feedback session with a practitioner using nine randomly selected examples to evaluate the usefulness of the added explainability support to the ML techniques for determining the validity of bug reports. On a scale of 1 to 5 (1 being not at all helpful and 5 being very helpful), the practitioner rated the SHAP-generated explanation as follows: (a) two samples received a rating of 5. (b) four samples received a rating of 4, (c) one sample received a rating of 3, and (d) two samples received a rating of 2.

Overall, the practitioner's response was also positive, and the practitioner commented that: "*The explanation for both predictions valid and invalid seems reasonable, and I can understand from these explanations why these bug reports are predicted as valid and invalid.*"

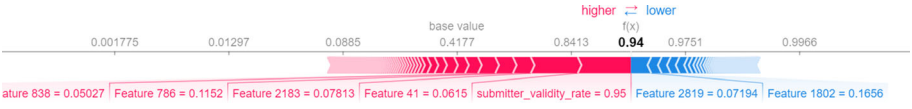


Fig. 16 Explanation of an individual instance of a valid bug report using force plot for XGBoost-based model, to maintain data confidentiality, the names of some company-specific features are anonymized

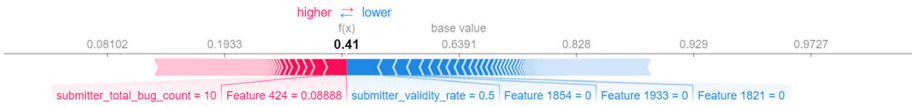


Fig. 17 Explanation of an individual instance of an invalid bug report using force plot for a logistic regression-based model, to maintain data confidentiality, the names of some company-specific features are anonymized

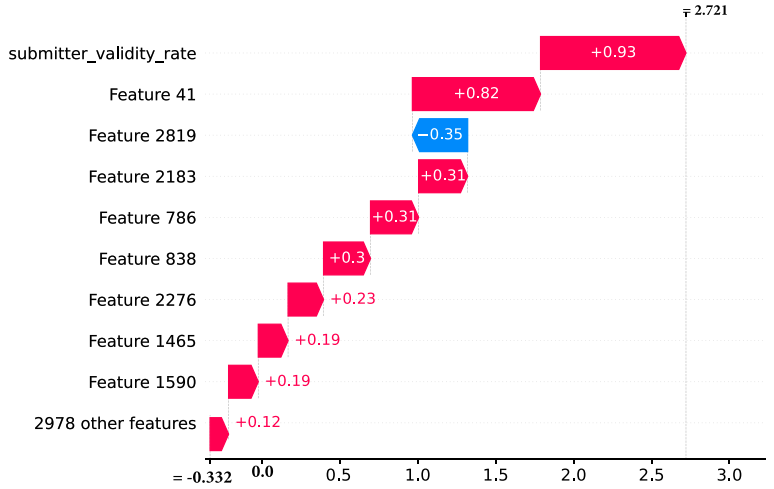


Fig. 18 Explanation of an individual instance of a valid bug report using waterfall plot for XGBoost-based model, to maintain data confidentiality, the names of some company-specific features are anonymized

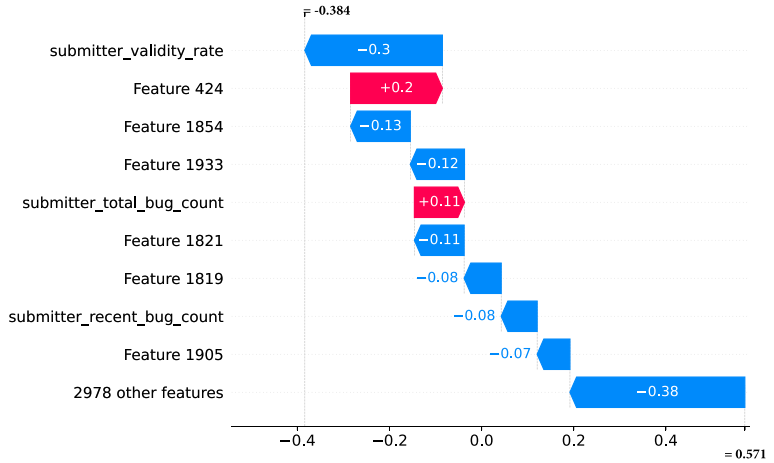


Fig. 19 Explanation of an individual instance of an invalid bug report using waterfall plot for the logistic regression-based model, to maintain data confidentiality, the names of some company-specific features are anonymized

Our evaluation involves a single practitioner and nine samples. This is a limitation of our study. However, other researchers have demonstrated SHAP's usefulness, even on a larger scale. For instance, Santos et al. (2020) used SHAP to explain software defect predictions to practitioners. In a survey of 40 developers, they found that most could understand the local explanation generated by SHAP. Additionally, SHAP has been extensively applied in various fields like medicine, banking, power, and irrigation with promising results (Moscato et al. 2021; Zhang et al. 2022, 2020; Jamei et al. 2024). Another indicator of the impact of the proposal (i.e., SHAP) is over 17k citations since its proposal in 2017.

Considering the existing research on SHAP (see above), evaluating the nine samples with the practitioner should be sufficient to demonstrate its use as a proof of concept in our context, i.e., its use in explaining the prediction of ML techniques to assess the validity of bug reports.

5.5 RQ5: How Large an Issue is Concept Drift in Bug Reports Data Used for Determining the Validity of Bug Reports?

To investigate concept drift in bug reports data used for determining the validity of bug reports, we followed an incremental approach with a 90-day window size similar to previous work (see Section 4.5). We applied both Logistic Regression and SVM classifiers and measured the significance of their prediction error rate on the old and recent windows. Table 9 presents the results of our analysis for the three products (see highlighted drift points in boldface text).

In product P1, during windows 3, 4, and 5, both the Logistic Regression and SVM have identified drift points in data. However, after that, the distribution shows no significant differences in the prediction error rate of old and recent windows until window 13, where we see another drift point. Afterward, the distribution is stable again, and the model performance does not degrade. In product P2, during window 2, both the Logistic Regression and SVM have identified a drift in data. The distribution is then stable until window 15 where we can see another drift point. SVM has identified one more drift point for window 19, and in window 22, both SVM and Logistic Regression detected a drift point. Likewise, for Product 3, the first drift has been identified during window 5 by the Logistic Regression, and the second and third drift points are identified for window 17 (i.e., by SVM only) and 18 (i.e., both the Logistic Regression and SVM have identified another drift point).

6 Discussion

In this Section, we discuss the findings of our study.

6.1 Need for Explainability and SHAP

During the initial validation in the industry, the practitioners indicated an interest in using the proposed ML technique for determining the validity of bug reports at early stages. However, their main concern was a lack of explanations behind predictions (i.e., why the predicted bug report is valid or invalid). These findings are in-line with other ML-based software analytics approaches (Dam et al. 2018) where one of the main challenges is to provide an explanation of the black box models. Nevertheless, we addressed this challenge using the SHAP framework (Lundberg and Lee 2017).

SHAP can help improve the explainability of predictions made by ML techniques. However, it has some limitations, which we discuss below:

Table 9 Concept drift when using logistic regression (LR) and support vector machines (SVM) on products P1, P2, and P3

Window	P1		P2		P3	
	p-value (LR)	*p-value* (SVM)	*p-value* (LR)	*p-value* (SVM)	*p-value* (LR)	*p-value* (SVM)
1	0.8693	0.6201	1.000	1.000	0.3627	0.3090
2	0.3662	0.2341	**0.0075**	**0.0206**	1.000	1.000
3	**0.0010**	**0.00006**	1.000	0.8344	1.000	1.000
4	**0.0017**	**0.0002**	0.0989	0.1918	0.6431	0.8252
5	**0.0138**	**0.0244**	0.5381	0.6751	**0.0022**	0.1603
6	0.0820	0.0820	0.7532	0.8769	0.5146	0.2936
7	0.7960	0.6985	0.8807	0.8826	1.000	0.8367
8	0.2933	0.1882	0.8828	1.000	1.000	0.8190
9	0.1857	0.1857	0.6887	0.6894	1.000	1.000
10	0.6439	0.6536	0.6024	0.7921	0.5740	1.000
11	0.0824	0.2371	0.8257	0.7416	0.7598	1.000
12	0.5020	0.5947	0.5012	0.9121	0.2437	0.3651
13	**0.0280**	**0.0280**	0.5507	0.6355	0.4793	0.3173
14	0.1622	0.1635	0.4181	0.4964	0.1983	0.1983
15	0.3143	0.5619	**0.0009**	**0.0003**	0.2487	0.4921
16	0.2989	0.1644	0.0653	0.1041	0.2049	0.2061
17	0.1656	0.1249	0.3035	0.0661	0.0710	**0.0137**
18	0.5602	0.5608	0.6262	0.9069	**0.0178**	**0.0178**
19	0.1670	0.0955	0.1918	**0.0175**	0.2704	0.2723
20	0.4502	0.1949	1.000	0.8122	0.0829	0.1407
21	0.0733	0.2299	0.0722	0.0769	0.0935	0.0594
22	0.7061	0.7061	**0.0442**	**0.0130**	0.6225	0.6225
23	0.8512	1.000	0.3606	0.5823	0.4778	0.1631

P-values < 0.05 are significant (highlighted in boldface)

- In our study, we found that SHAP-generated explanations may not always be helpful for practitioners. On a scale from 1 to 5 (1 being not at all helpful and 5 being very helpful), out of 9 instances, 2 received a rating of 2 out of 5, and 1 received a rating of 3 out of 5.
- SHAP can be computationally expensive for large datasets or complex models, e.g., pixel data (Mosca et al. 2022). In our study, we have used simple ML techniques on around 3300 bug reports data, and we did not observe such an issue. We were able to run our experiments on a machine equipped with Windows 10, 64-bit as the operating system, Intel (R) Core (TM) i7, and 16GB RAM.
- Slack et al. (2020) showed that explanations generated by LIME and SHAP could not conceal discriminatory biases of black-box classifiers in sensitive domains, e.g., criminal justice and credit scoring. They also proposed and assessed a novel framework that can conceal such biases.
- The assumptions made by SHAP can sometimes lead to the loss of important information, such as dependency relationships between features (Mosca et al. 2022). For example, the

symmetry property of Shapely values treats features as independent, which can only be true in some cases. Several approaches have been proposed for incorporating causal dependencies (e.g., Frye et al. 2020; Heskes et al. 2020).
– SHAP uses Shapely values to generate explanations for a model and its predictions. However, research has shown that the approach used by Shapely values to calculate feature importance is not suitable for quantifying feature importance in general (Kumar et al. 2020).
– SHAP uses Shapely values to generate explanations for a model and its predictions. However, Kumar et al. (2020) argue that these explanations are not always easy to understand and do not match human expectations, i.e., we usually want explanations that make sense to us as humans.

6.2 Need for Not Mislabeling Valid Bug Reports as Invalid

It is important to note that during the initial validation in the industry, practitioners highlighted that the ML technique should carefully handle the prediction of false negatives (i.e., false invalid bug reports) due to the implications for bug management at the case company. Predicting an invalid bug report as valid could be fine because it will not be impacted by subsequent actions such as deprioritization. However, if a valid bug is predicted to be invalid and such action is taken, it will have negative consequences. For instance, delaying the resolution of the valid bug could lead to other problems, such as further bugs in the system and customer dissatisfaction.

To handle the aforementioned situations, one solution we proposed, which practitioners also acknowledged, was accepting predictions for invalid bug reports when the ML technique is at least 90% confident.

6.3 Potential Uses of the Predictions of Bug Report Validity

The findings from in-use validation indicate the potential of the proposed solution for supporting practitioners in identifying invalid bug reports during the bug management process. The proposed solution aims to identify invalid bug reports early during the bug management process. Early identification here means that as soon as a new bug report arrives in a bug tracking system and before it is assigned to a relevant developer/team. Early identification of invalid bug reports has practical implications. For instance, if the validity of a newly submitted bug report is evaluated before the bug triage process begins (i.e., as shown in Fig. 20 before the change control board (CCB) screening). In that case, it will significantly help practitioners to effectively use developers' resources by prioritizing the assignment of developers to bug reports that are likely to be valid.

As shown in Fig. 20, the dotted line is used for the ML technique's suggestion to the CCB, development teams, or developers in the decision-making process. Additionally, as suggested by several practitioners, the ML technique's suggestion could also be used to provide feedback during the bug report registration stage.

The ML technique's goal is to assist the CCB and other stakeholders downstream in the bug management process by indicating the likely validity of a bug report. However, this is still only decision support, e.g., the CCB still has to decide on subsequent actions for the bug report.

In this study, we only conducted an initial in-use validation of the proposed approach to predict the validity of newly submitted bug reports on a single product, which is a limitation.

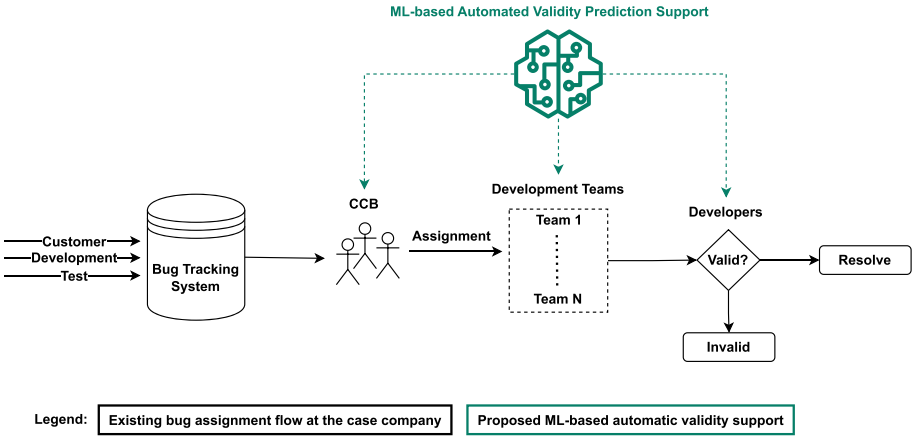


Fig. 20 Existing bug assignment flow at the case company (in black) and proposed automatic validity support (in green), adopted from Laiq et al. (2022)

6.4 Reflections on the Use of the Technology Transfer Model for Industrial Adoption of an ML Technique

Transferring solutions from academia to industry is challenging, and several approaches for successful technology transfer have been proposed (Daun et al. 2023). In this study, we used Gorschek et al. (2006)'s technology transfer model to guide the adoption of an ML technique at a company (see Fig. 1). Following the model, we systematically developed, evaluated, and iteratively improved our proposed solution based on the practitioners' feedback. The iterative approach, with several degrees of validation and involvement of practitioners in various steps, helped practitioners gain confidence in the proposed ML-based solution. This confidence is evident in one of the teams using the ML technique in their operations. By facilitating the actual use of the techniques in practice, the model enabled us to identify additional necessary conditions for operationalizing the proposed ML-based solution, such as the need for explainability and maintaining the accuracy of the ML technique over time.

6.5 Concept Drift in the Context of Predicting the Validity of Bug Reports

Our findings from the concept drift analysis indicate the presence of drift points in bug reports data used to determine the validity of bug reports (see Section 5.5). The drift points in the data will likely affect the prediction performance of the ML technique in its operational use. The performance degradation will have consequences on the use of the ML technique in practice for predicting the validity of bug reports. Thus, it is important to continuously monitor the model's performance, identify such drift points, and take necessary actions, such as retraining the model as soon as recent data is available and cautiously considering the current prediction.

To deal with concept drift in the context of predicting the validity of bug reports using ML techniques, the following strategies can be used:

– **Monitoring and alerting:** The performance of an ML technique should be continuously monitored, and alerts should be sent to relevant people about the performance and presence of concept drift. Based on the significance of the error rate in the performance of

the technique, software quality assurance teams should take relevant actions, such as retraining the technique's model as soon as new data is available and consider the current prediction with caution.

- **Regular model updates:** After the model is deployed, it is necessary to update it to ensure that it reflects recent trends in the data (Paleyes et al. 2022), that is, the ML model used for predicting the validity of bug reports should be updated at regular intervals to adapt to changing data distributions. However, simply updating the existing model may not always be sufficient to handle the concept drift. There might be a need for a new model or tuning to the existing model to handle concept drift, which is time-consuming, resource-intensive, and requires expertise. Thus, there is a need for an approach that can automatically find a suitable model for handling concept drift. A growing field of automated machine learning may be suitable for addressing this challenge (Hutter et al. 2019).
- **Use of topic modeling and similarity measuring techniques:** Topic modeling and similarity measuring techniques can be used to highlight the presence of concept drift in newly reported bug reports. In the case of topic modeling, topics can be generated from past data of bug reports, and newly reported bug reports can be classified into existing topics. If the newly submitted bug reports do not match any topic with a defined similarity threshold, it could indicate concept drift. Likewise, similarity measuring techniques can be used to find similar past bug reports for a newly submitted bug report to identify whether the submitted bug report is related to old or new concepts.

Impact of data size on concept drift detection: In this section, we discuss the potential impact of data size on detecting concept drift. As shown in Fig. 8, similar to previous studies (Zliobaite 2010; Kabir et al. 2019), we used an incremental window-based approach with a full-memory training strategy. The approach used might be affected by the data size. However, based on the results shown in Table 9, the impact of data size on concept drift (CD) detection seems unlikely. For example, for P1, we identified CD during windows 3,4,5 and 13, that is, for both cases when the data size is small and when the data size is large. Similarly, for P2, we identified CD during windows 2, 15, and 22.

6.6 Overall Implication

In this study, we used simple techniques to assess the validity of newly submitted bug reports at an early stage. Simple ML techniques were chosen for their evaluation in the industry for the early identification of invalid bug reports considering the following factors:

(a) Practitioners wanted to use simpler ML techniques as long as they delivered comparable results. Previous research (He et al. 2020) and our investigation (see Table 8) found that simple ML techniques achieved comparable results to the deep learning technique, fine-tuned BERT and the techniques based on DistilBERT embeddings.

(b) Using simple ML techniques, we achieved comparable results, i.e.,(He et al. 2020)'s deep learning approach achieved an average AUC of 0.85, while we achieved 0.82. Furthermore, He et al. (2020) also reported that their DL technique could not outperform (Fan et al. 2018)'s simple ML techniques for all projects, i.e., (Fan et al. 2018)'s simple ML technique achieved an AUC of 0.92 compared to He et al. (2020)'s deep learning technique for one project, which achieved 0.86 AUC.

(c) One of the main reasons for choosing simple ML techniques was the adoption challenges of complex techniques (Paleyes et al. 2022), e.g., long-term maintenance and steep learning curve. Borg et al. (2022) also reported similar observations for adopting an ML-based

tool in practice for automatic bug assignment, i.e., practitioners prefer a simple solution over a complex one that can deliver reasonably good performance. A simple model could help to demonstrate the concept of the proposed ML solution and get the end-to-end setup in place easily (Paleyes et al. 2022), which also helps in long-term maintenance (Borg et al. 2022). Haldar et al. (2019) reported that at Airbnb, they faced several issues and wasted several cycles in setting up a complex deep learning-based model. After several failed attempts, they eventually had to simplify it. Only after simplifying their deep learning model they could put it into the production pipeline.

Considering the adoption challenges (mentioned above) of ML techniques in practice and the requirement of practitioners, we wanted to keep our solution simple, i.e., simple techniques that produce results with comparable accuracy to complex techniques such as deep learning. Thus, we opted for simple ML techniques.

The results of both initial and in-use validation indicate the proposed solution's potential, i.e., the ML technique for the early identification of invalid bug reports. Overall, the feedback from the practitioners was positive.

6.7 Takeaways from the Industrial Adoption of ML Techniques for Early Identification of Invalid Bug Reports

Below, we describe takeaways from our investigation of the industrial adoption of ML techniques for the early identification of invalid bug reports.

Use simpler technique: We found that simple ML techniques produce comparable results to deep learning techniques and techniques based on BERT for identifying invalid bug reports. Furthermore, the simpler techniques are easier to understand and show additional advantages like shorter training time and require fewer computational resources. These advantages make simpler techniques more appropriate for practical adoption.

Additionally, in industry-academia collaboration, practitioners may have requirements to use simple ML techniques that can deliver comparable results to complex techniques and offer additional advantages, such as shorter training time and fewer computational resources. For instance, the practitioners of the case company where we studied the problem of identifying invalid bug reports at an early stage had such requirements. Thus, researchers should prioritize simple ML techniques in industry-academia collaboration to improve the likelihood of adopting ML techniques.

Explainability improves trustworthiness: Similar to other research (see Section 6.1), in the context of identifying invalid bug reports using ML techniques, we found that explainability is important for practitioners, and it improves the trustworthiness of an ML-based solution. Therefore, when designing ML-based solutions to support practitioners in their daily activities, we suggest providing explainability support to improve the likelihood of adopting ML techniques in practice.

False negatives are more critical than false positives in the context of predicting the validity of bug reports: In the context of identifying invalid bug reports using ML techniques, we found that mislabeling valid bug reports as invalid is more critical than the opposite due to its practical implications. Therefore, while designing ML-based solutions, it is important to carefully consider and consult with practitioners on handling false negatives and positives.

Non-disruptive integration is important: During our interactions with practitioners and through survey responses, we found that several practitioners stressed the need for smooth integration of the ML-based tool for the early identification of invalid bug reports. They

emphasized that the tool should seamlessly integrate with existing systems without disrupting the user and workflow. Therefore, it is important to consider the integration factor when designing ML-based tools to support practitioners in their daily tasks to increase the likelihood of adoption in practice. For instance, to avoid compatibility issues with existing systems at a company, the selection of an ML framework/library should be done in consultation with practitioners.

Be aware of concept drift: After deploying an ML model, it is possible that its performance may decline over time due to concept drift. In our experience of using an ML technique to identify invalid bug reports, we noticed performance degradation due to the presence of the concept during the in-use validation in the industry (see Section 6.5). Therefore, we recommend taking preventive measures when using ML-based solutions to support practitioners in software engineering tasks. For instance, the model's performance should be continuously monitored to detect any concept drift, and regular updates to the model should be made to maintain its performance over time.

Tool usability is a key factor for adoption: In the context of identifying invalid bug reports using an ML-based tool, we found that usability is an essential factor in improving the likelihood of adopting the tool. Several practitioners highlighted that the ML-based tool for identifying invalid bug reports should be easy to use, and usability is key for adoption. Furthermore, they also highlighted that the tool should have a self-explanatory user interface. Therefore, when designing ML-based tools to support practitioners in their daily activities, we recommend prioritizing the usability of the tools to improve their likelihood of adoption.

A step-by-step approach guided by the technology transfer model helps build trust and support for a proposed solution: In our study, we found that the adopted model for technology transfer helps build trust and support for a proposed solution, i.e., the iterative approach, with several degrees of validation and involvement of practitioners in various steps, helped practitioners gain confidence in the proposed ML-based solution. For instance, in our context, this confidence is evident in one of the teams at the case company that used the ML-based solution in their operations for in-use validation.

Thus, (a) for practitioners to gain trust in an ML-based solution, we suggest adopting a similar approach, and (b) For researchers in industry-academia collaboration, we suggest following a similar approach to improve the likelihood of adopting an ML-based solution in practice.

A step-by-step approach guided by the technology transfer model helps identify the necessary conditions for operationalizing an ML-based solution: In our study, we found that the adopted model for technology transfer helps identify additional necessary conditions for operationalizing an ML-based solution. For instance, in the context of identifying invalid bug reports using an ML technique, the model helped us to identify several necessary conditions for operationalizing an ML-based solution, e.g., the need for explainability and maintaining the accuracy of the ML technique over time.

Thus, for both practitioners and researchers, we suggest adopting a similar approach to identify further conditions necessary for operationalizing an ML-based solution.

7 Validity Threats

In this section, we describe the potential validity threats of our study.

Generalizability of Results We consider the case company to be a typical case of large-scale closed-source software development. We applied an ML technique to three of their products (i.e., one large, one medium, and one small) with around 10K bug reports. Furthermore, the results of the applied ML technique are comparable to those from related work in the OSS context. These aspects mentioned above give us confidence that ML techniques for validity prediction will likely generalize to similar contexts.

Small Sample Sizes Sample sizes in all evaluations are small. The feedback from a small sample of practitioners during focus group meetings could threaten generalizability within the company and externally. However, these practitioners represent diverse roles at the case company and have substantial expertise.

Similarly, the identification of practitioners' adoption concerns, such as the explainability and integration with the existing toolchain, is based on a relatively small number of responses (i.e., 34), which could threaten the generalizability of these results within the company (12 out of 34 responses belong to the case company) and externally. However, these practitioners are highly experienced, represent diverse roles, and belong to different companies (i.e., small, medium, and large).

The feedback on the usefulness of the applied SHAP framework (Lundberg and Lee 2017) to explain the prediction results of the ML technique was evaluated with a single practitioner, but still valuable as the evaluation was done with a senior architect who has an in-depth understanding of the product and the technology. Furthermore, other researchers (see Section 5.4) have demonstrated SHAP's usefulness, even on a larger scale. Thus, we believe evaluating the nine samples with a single practitioner should be sufficient to demonstrate its use as a proof of concept in our context.

Observer Bias The feedback collected during the focus group meetings might be influenced by observer bias. To mitigate observer bias, two researchers participated in each focus group meeting. One led the discussion, and the other took notes. Afterward, researchers discussed and documented their interpretations. We also performed member checking by presenting our findings and analysis back to practitioners.

Reliability of Explanations of the ML Technique's Predictions To have reliable explanations for the ML technique's predictions, we used a theoretically sound existing explainability framework by Lundberg and Lee (2017), please see Section 5.4 for details.

Relevance of Technology Adoption Concerns To identify relevant adoption concerns of practitioners for adopting the ML technique to identify invalid bug reports at an early stage, we used an existing framework by Rana et al. (2014b).

Reliability of Training Data The training data used in this study only contains the bug reports with a finished status (i.e., validity labels: valid or invalid), as these labels represent judgments of more than one domain expert.

The training data used by ML techniques could be impacted by concept drift. We investigated and identified the impact of concept drift on the performance of the ML technique for the early identification of invalid bug reports (see Section 5.5).

The training data was based on invalid bug reports that had assigned labels. However, a bug report could potentially have been updated before it was assessed for validity. For example, a submitter might have added an attachment or a clarification. Therefore, we should have used the initial versions of the bug reports for model training since the ML model will be applied to the initial versions of the new bug reports. In practice, though, we only had access to the

final state of the bug reports. The model was, therefore, trained on the final stages of bug reports and used to assess early-stage descriptions of bug reports. This is a limitation of the study. However, we do not consider that a major threat to the study's validity since, at the case company, the bug reports go through an initial review for completeness before being assessed for validity by the ML technique.

Validity of the Evaluation Measures As the bug reports data has class imbalance (a significantly larger number of valid bug reports compared to invalid ones), we used two robust measures, AUC and MCC, to evaluate the performance of the ML techniques. These measures were used as they provide a reliable assessment when the data has class imbalance (see Section 4.2.7 for a detailed discussion).

8 Conclusion and Future Work

In this study, we used a technology transfer model to guide the adoption of the ML technique for the early identification of invalid bug reports at a large-scale software development company. We conducted an initial in-use validation of the proposed technique in a real industrial setting on one of the products at the company (see Section 2.3 for details about the contribution of this extension paper).

The results of our study indicate that the ML technique can identify invalid bug reports at early stages with acceptable accuracy and has the potential to assist software maintenance practitioners in managing bug reports effectively. Based on the ML technique's recommendation, they can prioritize the resolution of valid bug reports and reduce the effort spent on invalid bug reports.

We found that providing an explanation of the ML technique's predictions helps improve the technique's trustability and the likelihood of adoption. Likewise, maintaining the accuracy of the ML technique over time and smooth integration into the existing toolchain will also improve the likelihood of adoption.

In the future, we aim to evaluate our proposed technique for a longer time and on more data and possibly other products at the company. Furthermore, we aim to evaluate the proposed technique in other companies where managing invalid bug reports is challenging.

A Implementation Details

A.1 Hyperparamters for each Model of the ML Techniques

Below, we provide the hyperparameters for each model that were selected after performing tuning using the GridSearchCV (scikit-learn developers 2023) supported by sklearn, while the rest were kept at their default values:

- Logistic Regression hyperparameters for data from product P1: {'C': 0.1, 'penalty': 'l2', 'solver': 'saga'}
- SVM hyperparameters for data from product P1: {'C': 10, 'gamma': 0.01, 'kernel': 'sigmoid'}
- Random Forest hyperparameters for data from product P1: {'max_depth': 9, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}

– XGBoost hyperparameters for data from product P1: {'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50}
– Logistic Regression hyperparameters for data from product P2: {'C': 0.01, 'penalty': 'l1', 'solver': 'liblinear'}
– SVM hyperparameters for data from product P2: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
– Random Forest hyperparameters for data from product P2: {'max_depth': 9, 'min_samples _leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}
– XGBoost hyperparameters for data from product P2: {'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50}
– Logistic Regression hyperparameters for data from product P3: {'C': 10, 'penalty': 'l2', 'solver': 'liblinear'}
– SVM hyperparameters for data from product P3: {'C': 1, 'gamma': 'scale', 'kernel': 'linear'}
– Random Forest hyperparameters for data from product P3: {'max_depth': 9, 'min'_samples _leaf': 1, 'min_samples_split': 10, 'n_estimators': 200}
– XGBoost hyperparameters for data from product P3: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 50}

A.2 Implementation details of CNN, BERT, and DistilBERT

CNN: For word embedding, we used word2vec using the Skip-gram algorithm. In the input layer, we used a truncation strategy to unify the input length, i.e., we set the length of words to max 60 for textual features. In the convolution layer, we use three kernels with the activation function rectified linear unit (RELU). In the pooling layer, global max-pooling is adopted. In the fully connected layer, we use the sigmoid as an activation function and adam as an optimizer, and the learning rate and epoch were set: 0.01 with epoch 4 for P1 and P3 and 0.001 with epoch 5 for P2.

BERT: To fine-tune BERT for our task, we used the 'bert-base-uncased' pre-trained model (Devlin et al. 2018) and BertTokenizer from the Hugging Face's transformers library[3]: (i) BertTokenizer.from_pretrained('bert-base-uncased') and (ii) BertForSequenceClassification.from_pretrained('bert-base-uncased'). To encode data (i.e., converting data into the required format by a pre-trained BERT model), we used the batch_encode_plus method from BertTokenizer. We fine-tuned the pre-trained BERT model for four epochs and used AdamW as an optimizer.

DistilBERT: We used DistilBertTokenizer and DistilBertModel from transformers for word embedding. We then use the embedded vector as a feature vector with the two top-performing simple ML techniques, i.e., Logistic Regression (DistilBERT) and SVM (DistilBERT).

For Logistics Regression (DistilBERT), we use the following hyperparameters that were selected after performing tuning using the GridSearchCV (scikit-learn developers 2023) supported by sklearn, while the rest were kept at their default values:

[3] https://www.huggingface.co/docs/transformers/

– Logistic Regression hyperparameters for data from product P1: {'C': 0.1, 'penalty': 'l1', 'solver': 'liblinear'}
– Logistic Regression hyperparameters for data from product P2: {'C': 0.1, 'penalty': 'l1', 'solver': 'liblinear'}
– P3: Logistic Regression hyperparameters for data from product P3: {'C': 0.1, 'penalty': 'l2', 'solver': 'lbfgs'}

For SVM (DistilBERT), we use the following hyperparameters that were selected after performing tuning using the GridSearchCV (scikit-learn developers 2023) supported by sklearn, while the rest were kept at their default values:

– SVM hyperparameters for data from product P1: {'C': 1, 'gamma': 0.001, 'kernel': 'rbf'}
– SVM hyperparameters for data from product P2: {'C':0.1, 'gamma': 0.001, 'kernel': 'rbf'}
– SVM hyperparameters for data from product P3: {'C': 10, 'gamma': 0.001, 'kernel': 'rbf'}

B Detailed Results

In Table 10, we present the detailed results of the applied ML techniques using features F1–F5, as reported in our previous work (Laiq et al. 2022). In Table 11, we present the detailed results of the applied ML techniques using features F1–F13. Details about feature selection are described in Section 4.2.4. The complete set of features (F1–F13) is described in Table 4.

Table 10 Model performance using the features F1–F5: Area under the curve (AUC), nMCC, F1-score (F1), precision (P), and recall (R) for valid (V) and invalid (I) bug reports

Product	Classifer	AUC	nMCC	F1(I)	P(I)	R(I)	F1(V)	P(V)	R(V)
P1	LR	**0.851**	0.705	0.411	0.705	0.305	0.942	0.906	0.982
	SVM	**0.849**	0.657	0.303	0.574	0.220	0.940	0.896	0.989
	RF	0.681	0.560	0.161	0.320	0.120	0.919	0.882	0.960
	XGB	**0.817**	0.672	0.313	0.742	0.204	0.939	0.894	0.989
P2	LR	0.765	0.614	0.247	0.538	0.168	0.910	0.857	0.970
	SVM	0.753	0.608	0.166	0.727	0.099	0.914	0.849	0.990
	RF	0.619	0.536	0.106	0.293	0.069	0.900	0.840	0.971
	XGB	0.739	0.596	0.187	0.570	0.124	0.908	0.851	0.973
P3	LR	**0.840**	0.735	0.530	0.725	0.448	0.902	0.872	0.938
	SVM	**0.839**	0.740	0.529	0.754	0.426	0.909	0.868	0.955
	RF	0.627	0.535	0.104	0.28	0.075	0.873	0.804	0.959
	XGB	**0.813**	0.702	0.458	0.720	0.366	0.892	0.852	0.938

Classifiers: logistic regression (LR), support vector machine (SVM), random forest (RF), and XGBClassifier (XGB)
Top AUC values are highlighted in boldface

Table 11 Model performance using features F1–F13: Area under the curve (AUC), nMCC, F1-score (F1), precision (P), and recall (R) for valid (V) and invalid (I) bug reports

Product	Classifer	AUC	nMCC	F1(I)	P(I)	R(I)	F1(V)	P(V)	R(V)
P1	LR	**0.845**	0.679	0.395	0.6685	0.290	0.941	0.904	0.982
	SVM	**0.835**	0.684	0.367	0.629	0.269	0.940	0.901	0.985
	RF	0.682	0.574	0.186	0.314	0.140	0.923	0.885	0.966
	XGB	**0.817**	0.672	0.313	0.742	0.204	0.939	0.894	0.989
P2	LR	0.757	0.615	0.252	0.528	0.171	0.909	0.857	0.969
	SVM	0.716	0.605	0.139	0.816	0.078	0.915	0.846	0.996
	RF	0.627	0.536	0.105	0.278	0.067	0.90	0.841	0.973
	XGB	0.739	0.596	0.187	0.470	0.124	0.908	0.851	0.973
P3	LR	**0.834**	0.745	0.561	0.717	0.492	0.897	0.877	0.922
	SVM	**0.834**	0.752	0.563	0.747	0.476	0.906	0.876	0.941
	RF	0.638	0.520	0.092	0.192	0.069	0.871	0.805	0.956
	XGB	**0.816**	0.704	0.448	0.747	0.354	0.893	0.851	0.944

Classifiers: logistic regression (LR), support vector machine (SVM), random forest (RF), and XGBClassifier (XGB)

C Questionnaire for Identifying Adoption Factors of Machine Learning Techniques

Survey Context

In practice, bug reports are submitted to describe erroneous behaviors of a software system. However, a submitted bug report may not describe an issue in the code. Such bug reports are called 'invalid.' Invalid bug reports could be related to incorrect configurations, updates to documentation, feature requests, or misunderstandings of the system's functionality. The handling of invalid bug reports will consume time and effort as they are generally handled in the same way as other (valid) bug reports. They also often lead to the misallocation of developers' resources. To reduce the effort spent on invalid bug reports, we propose a machine learning-based tool to identify invalid bug reports at an early stage (i.e., as soon as a new bug report arrives in a bug tracking system and before it is assigned to a relevant developer/team).

This questionnaire aims to gather your personal opinion on the perceived usefulness and the likelihood of adopting an ML-based tool for the early identification of invalid bug reports at your company.

Invalid bug reports: The bug reports that do not describe issues in the code.
Early identification of invalid bug reports: As soon as a new bug report arrives in a bug tracking system and before it is assigned to a relevant developer/team.

Questions

Q1: In your opinion, how important is the early identification of invalid bug reports? Choose one of the following answers.

- Essential (must have)
- Worthwhile (nice to have)
- Unimportant
- Unwise (detraction)

Q2: In your opinion, how important is tool support for the early identification of invalid bug reports? Choose one of the following answers.

- Essential (must have)
- Worthwhile (nice to have)
- Unimportant
- Unwise (detraction)

Q3: Which of the following use cases are likely relevant at your company for an ML-based tool to identify invalid bug reports early? Select all that apply.

- As input for prioritizing bug reports
- Just to flag likely invalid bug reports for downstream developers
- To identify candidate bug report for an additional review
- I do not see any use case for the tool at our company
- There are other uses for the tool (not listed here), If yes, then we ask: Please briefly state what other use cases you foresee for such a tool.

Q4: How important are the following factors at your company for adopting an ML-based tool for the early identification of invalid bug reports? Not at all important (1) Slightly unimportant (2) Neutral (3) Important (4) Very important (5).

- High accuracy of the predictions
- Explainability of the predictions
- Adaptability to different products/projects
- Ability to handle a large amount of data
- Easy integration into the current workflow
- Easy integration with existing systems
- Compatibility with existing systems
- Low cost for data collection
- Low cost for training and education of people
- Low cost for installation
- Low cost for licensing
- Low cost for computational resources
- Low cost for maintenance
- Short training time
- Low learning curve
- Buy-in from relevant stakeholders
- Availability as an open-source tool
- Availability of technical support

Q5: What other factors do you think are essential to enable the adoption of such a tool besides those listed in the previous question? Free text.
Q6: How familiar are you with machine-learning (ML) techniques and solutions? Select all that apply.

- I have collaborated with academics in projects where we used ML
- I have tried ML using existing libraries (e.g., scikit-learn) in previous projects

– I have interpreted the results of ML techniques
– I have assessed the quality of the results of ML techniques
– I have implemented ML techniques from scratch in-house
– I am familiar with ML for reasons other than those listed above
– I am not familiar with ML

Q7: In your opinion, how familiar are potential users at your company (e.g., bug assigners, quality assurance people, and developers) with machine-learning (ML) techniques and solutions? Select all that apply.

– They have collaborated with academics on projects where they have used ML
– They have tried ML using existing libraries (e.g., scikit-learn) in previous projects
– They have interpreted the results of ML techniques
– They have assessed the quality of the results of ML techniques
– They have implemented ML techniques from scratch in-house
– They are familiar with ML for reasons other than those listed above
– They are not familiar with ML

Q8: How many years of professional experience do you have in the software industry?
Q9: Could you please briefly describe the size and type of your company?
Q10: Could you please briefly describe your role at your company?
Q11: Are your job responsibilities related to bug management, software maintenance, or software quality? [Yes, No].

Acknowledgements This work has been supported by ELLIIT, a Strategic Research Area within IT and Mobile Communications funded by the Swedish Government, and the GIST project (ref# 20220235) from the Knowledge Foundation, Sweden.

Funding Open access funding provided by Blekinge Institute of Technology.

Data Availability The data supporting this study's findings are not openly available due to restrictions from the case company.

Declarations

Conflicts of Interests The authors declared that they have no conflict of interest.

References

Aktas EU, Yilmaz C (2020) Automated issue assignment: results and insights from an industrial case. Empirical Software Engineering 25(5):3544–3589

Bachmann A, Bernstein A (2009) Software process data quality and characteristics: a historical view on open and closed source projects. In: The joint international and annual ERCIM workshops on Principles of Software Evolution (IWPSE) and software evolution (Evol) workshops, pp 119–128

Bennin KE, Ali NB, Börstler J, Yu X (2020) Revisiting the impact of concept drift on just-in-time quality assurance. 20th International Conference on Software Quality. Reliability and Security, QRS, IEEE, pp 53–59

Bettenburg N, Premraj R, Zimmermann T, Kim S (2008) Duplicate bug reports considered harmful. . . really? In: International conference on software maintenance, IEEE, pp 337–345

Bhattacharya P, Neamtiu I, Shelton CR (2012) Automated, highly-accurate, bug assignment using machine learning and tossing graphs. Journal of Systems and Software 85(10):2275–2292

Biswas E, Karabulut ME, Pollock L, Vijay-Shanker K (2020) Achieving reliable sentiment analysis in the software engineering domain using bert. In: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 162–173

Borg M, Jonsson L, Engström E, Bartalos B, Szabó A (2022) Adopting automated bug assignment in practice: a longitudinal case study at ericsson. arXiv:2209.08955

Borg M, Runeson P, Ardö A (2014) Recovering from a decade: a systematic mapping of information retrieval approaches to software traceability. Empirical Software Engineering 19(6):1565–1616

Bradley AP (1997) The use of the area under the roc curve in the evaluation of machine learning algorithms. Pattern Recognition 30(7):1145–1159

Dam HK, Tran T, Ghose A (2018) Explainable software analytics. In: Proceedings of the 40th international conference on software engineering: new ideas and emerging results, pp 53–56

Daun M, Brings J, Aluko Obe P, Tenbergen B (2023) An industry survey on approaches, success factors, and barriers for technology transfer in software engineering. Practice and Experience, Software

de Lima Cabral DR, de Barros RSM (2018) Concept drift detection based on fisher's exact test. Information Sciences 442:220–234

Devlin J, Chang MW, Lee K, Toutanova K (2018) Bert: pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805

Ekanayake J, Tappolet J, Gall HC, Bernstein A (2012) Time variance and defect prediction in software projects. Empirical Software Engineering 17(4):348–389

Fan Y, Xia X, Lo D, Hassan AE (2018) Chaff from the wheat: characterizing and determining valid bug reports. IEEE Transactions on Software Engineering 46(5):495–525

Frye C, Rowat C, Feige I (2020) Asymmetric shapley values: incorporating causal knowledge into model-agnostic explainability. Advances in Neural Information Processing Systems 33:1229–1239

Gorschek T, Garre P, Larsson S, Wohlin C (2006) A model for technology transfer in practice. IEEE software 23(6):88–95

Haldar M, Abdool M, Ramanathan P, Xu T, Yang S, Duan H, Zhang Q, Barrow-Williams N, Turnbull BC, Collins BM et al (2019) Applying deep learning to airbnb search. In: Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining, pp 1927–1935

Halimu C, Kasem A, Newaz SS (2019) Empirical comparison of area under roc curve (auc) and Mathew correlation coefficient (mcc) for evaluating machine learning algorithms on imbalanced datasets for binary classification. In: 3rd international conference on machine learning and soft computing, pp 1–6

Heskes T, Sijben E, Bucur IG, Claassen T (2020) Causal shapley values: exploiting causal knowledge to explain individual predictions of complex models. Advances in Neural Information Processing Systems 33:4778–4789

He J, Xu L, Fan Y, Xu Z, Yan M, Lei Y (2020) Deep learning based valid bug reports determination and explanation. In: 31st International Symposium on Software Reliability Engineering (ISSRE), IEEE, pp 184–194

Huang J, Ling CX (2005) Using auc and accuracy in evaluating learning algorithms. IEEE Transactions on knowledge and Data Engineering 17(3):299–310

Hutter F, Kotthoff L, Vanschoren J (2019) Automated machine learning: methods, systems, challenges. Springer Nature

Jamei M, Ali M, Karbasi M, Karimi B, Jahannemaei N, Farooque AA, Yaseen ZM (2024) Monthly sodium adsorption ratio forecasting in rivers using a dual interpretable glass-box complementary intelligent system: hybridization of ensemble tvf-emd-vmd, boruta-shap, and explainable gpr. Expert Systems with Applications 237:121512

Kabir MA, Keung JW, Bennin KE, Zhang M (2019) Assessing the significant impact of concept drift in software defect prediction. In: 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC), IEEE, vol 1, pp 53–58

Kabir MA, Keung JW, Bennin KE, Zhang M (2020) A drift propensity detection technique to improve the performance for cross-version software defect prediction. In: 2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC), IEEE, pp 882–891

Klinkenberg R, Joachims T (2000) Detecting concept drift with support vector machines. In: ICML, pp 487–494

Kumar IE, Venkatasubramanian S, Scheidegger C, Friedler S (2020) Problems with shapley-value-based explanations as feature importance measures. In: International conference on machine learning, PMLR, pp 5491–5500

Laiq M, Ali Nb, Böstler J, Engström E (2022) Early identification of invalid bug reports in industrial settings–a case study. In: International conference on product-focused software process improvement, Springer, pp 497–507

Last M (2002) Online classification of nonstationary data streams. Intelligent Data Analysis 6(2):129–147

Lessmann S, Baesens B, Mues C, Pietsch S (2008) Benchmarking classification models for software defect prediction: a proposed framework and novel findings. IEEE Transactions on Software Engineering 34(4):485–496

Li Y, Jiang ZM, Li H, Hassan AE, He C, Huang R, Zeng Z, Wang M, Chen P (2020) Predicting node failures in an ultra-large-scale cloud computing platform: an aiops solution. ACM Transactions on Software Engineering and Methodology (TOSEM) 29(2):1–24

Linardatos P, Papastefanopoulos V, Kotsiantis S (2020) Explainable ai: a review of machine learning interpretability methods. Entropy 23(1):18

Lundberg SM, Lee SI (2017) A unified approach to interpreting model predictions. Advances in neural information processing systems 30

Matthews BW (1975) Comparison of the predicted and observed secondary structure of t4 phage lysozyme. Biochimica et Biophysica Acta (BBA)-Protein Structure 405(2):442–451

Mehta CR (1994) The exact analysis of contingency tables in medical research. Statistical Methods in Medical Research 3(2):135–156

Molnar C (2023) Shapley values. https://christophm.github.io/interpretable-ml-book/shapley.html, online; accessed 14 February 2023

Mosca E, Szigeti F, Tragianni S, Gallagher D, Groh G (2022) Shap-based explanation methods: a review for nlp interpretability. In: Proceedings of the 29th international conference on computational linguistics, pp 4593–4603

Moscato V, Picariello A, Sperlí G (2021) A benchmark of machine learning approaches for credit score prediction. Expert Systems with Applications 165:113986

Mustapha IB, Hasan S, Olatunji SO, Shamsuddin SM, Kazeem A (2020) Effective email spam detection system using extreme gradient boosting. arXiv:2012.14430

Nguyen T (2021) Cross-applicability of ml classification methods intended for (non-) functional requirements. Master's thesis, University of Twente

Nishida K, Yamauchi K (2007) Detecting concept drift using statistical testing. Discovery science, Springer 4755:264–269

OECD (2023) Enterprises by business size. https://data.oecd.org/entrepreneur/enterprises-by-business-size.htm?m=1, Accessed 28 Nov 2023

Oliveira P, Andrade RM, Barreto I, Nogueira TP, Bueno LM (2021) Issue auto-assignment in software projects with machine learning techniques. In: 2021 IEEE/ACM 8th International Workshop on Software Engineering Research and Industrial Practice (SER&IP), IEEE, pp 65–72

Paleyes A, Urma RG, Lawrence ND (2022) Challenges in deploying machine learning: a survey of case studies. ACM Computing Surveys 55(6):1–29

Rana R, Staron M, Berger C, Hansson J, Nilsson M, Meding W (2014a) The adoption of machine learning techniques for software defect prediction: an initial industrial validation. In: Joint conference on knowledge-based software engineering, Springer, pp 270–285

Rana R, Staron M, Hansson J, Nilsson M, Meding W (2014b) A framework for adoption of machine learning in industry for software defect prediction. In: 9th International Conference on Software Engineering and Applications (ICSOFT-EA), IEEE, pp 383–392

Ribeiro MT, Singh S, Guestrin C (2016) "Why should I trust you?" Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 1135–1144

Runeson P, Höst M (2009) Guidelines for conducting and reporting case study research in software engineering. Empir Softw Eng 14(2):131–164

Santos G, Figueiredo E, Veloso A, Viggiato M, Ziviani N (2020) Predicting software defects with explainable machine learning. In: Proceedings of the XIX Brazilian symposium on software quality, pp 1–10

scikit-learn developers (2023) Tf-idf. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html, Accessed 26 Nov 2023

Shahri N, Lai SBS, Mohamad MB, Rahman H, Rambli AB (2021) Comparing the performance of adaboost, xgboost, and logistic regression for imbalanced data. Math Stat 9:379–85

Slack D, Hilgard S, Jia E, Singh S, Lakkaraju H (2020) Fooling lime and shap: adversarial attacks on post hoc explanation methods. In: Proceedings of the AAAI/ACM conference on AI, ethics, and society, pp 180–186

Sun J (2011) Why are bug reports invalid? 4th international conference on software testing. Verification and Validation, IEEE, pp 407–410

Sun C, Qiu X, Xu Y, Huang X (2019) How to fine-tune bert for text classification? In: Chinese computational linguistics: 18th China national conference, CCL 2019, Kunming, China, 18–20 October 2019, proceedings 18, Springer, pp 194–206

Tantithamthavorn C, Hassan AE (2018) An experience report on defect modelling in practice: pitfalls and challenges. In: 40th International conference on software engineering: software engineering in practice, pp 286–295

Tantithamthavorn C, Hassan AE, Matsumoto K (2018) The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. IEEE Trans Software Eng 46(11):1200–1219

Wang S, Zhang W, Wang Q (2014) Fixercache: unsupervised caching active developers for diverse bug triage. In: 8th ACM/IEEE international symposium on empirical software engineering and measurement, pp 1–10

Witten IH, Frank E (2002) Data mining: practical machine learning tools and techniques with java implementations. Acm Sigmod Record 31(1):76–77

Wu J, Ye C, Zhou H (2021) Bert for sentiment classification in software engineering. In: 2021 International Conference on Service Science (ICSS), IEEE, pp 115–121

Xu Y, Sui K, Yao R, Zhang H, Lin Q, Dang Y, Li P, Jiang K, Zhang W, Lou JG et al (2018) Improving service availability of cloud systems by predicting disk error. In: 2018 USENIX Annual Technical Conference, pp 481–494

Zanetti MS, Scholtes I, Tessone CJ, Schweitzer F (2013) Categorizing bugs with social networks: A case study on four open source software communities. In: 35th International Conference on Software Engineering (ICSE), IEEE, pp 1032–1041

Zhang Y, Weng Y, Lund J (2022) Applications of explainable artificial intelligence in diagnosis and surgery. Diagnostics 12(2):237

Zhang K, Xu P, Zhang J (2020) Explainable ai in deep reinforcement learning models: a shap method applied in power system emergency control. In: 2020 IEEE 4th conference on energy internet and energy System Integration (EI2), IEEE, pp 711–716

Zliobaite I (2010) Learning under concept drift: an overview. http://arxiv.org/abs/1010.4784,1010.4784

Muhammad Laiq is currently pursuing his Ph.D. in software engineering at Blekinge Institute of Technology. His research interests revolve around empirical software engineering and software analytics. He has obtained a double master's degree in software engineering from the University of Oulu and the Technical University of Madrid. Currently, his research is focused on utilizing data-driven approaches to manage invalid bug reports.

Nauman bin Ali is a senior lecturer at Blekinge Institute of Technology. He is involved in empirical research in the field of software engineering. His research interests include data and visual analytics, Lean software development, software testing, software process simulation, software metrics, and evidence based software engineering. He received his Ph.D., (2015) in software engineering from Blekinge Institute of Technology, Sweden and his B.S. (2004) in computer science from National University of Computer and Emerging Sciences, Pakistan.

Jürgen Börstler is a professor of software engineering at Blekinge Institute of Technology (BTH), Sweden. He received a Ph.D. in computer science from Aachen University of Technology (RWTH), Germany. Prof. Börstler is a member of SERL-Sweden, the Software Engineering Research and Education Lab at BTH and a senior member of the Swedish Requirements Engineering Network. Furthermore, he is a founding member of the Scandinavian Pedagogy of Programming Network. His research interests are in behavioral software engineering, research methods, software process improvement, software quality, program comprehension, and computer science education.

Emelie Engström is an associate professor at Lund University, Sweden, and a member of the Software Engineering Research Group. She conducts research on software quality assurance in collaboration with industry, and in national and international research collaborations. Her research interests include empirical large scale software engineering, continuous monitoring, and testing of evolving software systems, testing of complex and autonomous systems, and methodological aspects of software engineering research and industry academia knowledge co-creation.